A Hybrid Asymmetric Password-Authenticated Key Exchange in the Random Oracle Model

Jelle Vos[1], Stanislaw Jarecki[2], Christopher A. Wood[1], Cathie Yun[1], Steve Myers[1], and Yannick Sierra[1]

[1] Apple, Inc.
[2] University of California Irvine

Abstract. Symmetric encryption allows us to establish a secure channel based on a shared, strong key. However, users cannot remember or cannot store such keys securely. Password-Authenticated Key Exchange (PAKE) protocols address this by using low-entropy, human-memorizable passwords to establish secure channels. PAKEs are widely used and are foundational in practical cryptographic protocols, but while cryptographic tools like Key Encapsulation Mechanism (KEM) and Signatures have been implemented to resist attacks from quantum computers, PAKEs have gained quantum security only recently.
To hedge against any potential vulnerabilities in recent quantum-secure PAKEs and in their implementations, we primarily focus on hybrid PAKE constructions that compose CPace, a classically-secure PAKE, with a variant of a recently proposed quantum-secure PAKE, which we call OQUAKE. Specifically we introduce and analyze two new hybrid PAKEs designed to be efficient, easy to implement, and utilize a minimized set of standard building blocks. The first, called CPaceOQUAKE, is a hybrid symmetric PAKE that remains secure as long as either a classical or post-quantum assumption holds. The second, called CPaceOQUAKE+, is a hybrid asymmetric PAKE (aPAKE) where the server party holds a verifier that obscures the password, instead of holding the password itself.
In our analysis we present the necessary security proofs in the Universal Composability framework. In particular, we prove that OQUAKE, the underlying KEM-based PAKE in our hybrid constructions, realizes a relaxed UC PAKE variant that exposes password equality to passive observers, an observation available anyway in typical applications of PAKEs where the network interactions which follow the PAKE depend on authentication success. Moreover, we prove that our variant of the PAKE(+KEM)-to-aPAKE compiler is a similarly relaxed UC aPAKE.

Keywords: Post-quantum cryptography · Password-authenticated key exchange · PAKE · aPAKE · KEM.

1 Introduction

Symmetric encryption allows us to realize efficient secure communication channels between two parties, as long as they share a strong key. In general, the

problem of key establishment and management is solved by a public key infrastructure (PKI), but many scenarios cannot assume PKI to be in place. In those scenarios, users would have to remember a key by heart or they would have to manually type it over. With secure keys being at least 128 bits in size, either task is practically impossible for the average user.

Password-Authenticated Key Exchange (PAKE) [16] allows us to establish a strong shared key from a low-entropy password instead. PAKE is a ubiquitous tool which is used in countless real-world cryptographic protocols, including pairing devices using a short code, and establishing secure encrypted channels between a user's device and a server.

Most PAKEs deployed today, such as SRP, SPAKE2+, and OPAQUE, are only secure against a classical (non-quantum) attacker. Only recently, there have been proposals for quantum-secure PAKEs, though these proposals have multiple shortcomings.

First, many post-quantum PAKE proposals relied on more experimental hardness assumptions such as isogenies [3,50,62], or they were significantly less efficient than classical PAKEs [62]. Following the *Encrypted Key Exchange* (EKE) approach of Bellovin and Merritt [16], a line of works investigated generic constructions of PAKE from any KEM, including the standardized post-quantum KEM like ML-KEM (Kyber) [67,22], which results in PAKEs with efficiency very close to the underlying KEM. However, until very recently these constructions [13,73,68,8,10] relied on the so-called *Ideal Cipher* model for security analysis, and it is currently not clear how to instantiate an ideal cipher in a quantum computation adversarial setting.

Secondly, until very recently, quantum-secure PAKEs also lacked another important practical property: they did not provide hybrid security. A hybrid PAKE is a PAKE that remains secure if either of two hardness assumptions suffices to secure the whole, i.e. *either* the current classical hardness assumptions, which underlie currently standardized PAKE such that SPAKE2 [5] or CPace [31,4], *or* the post-quantum assumptions like (decisional) Module LWE (MLWE), which underlies ML-KEM. Whereas running a classical and a post-quantum primitive in parallel yields a secure hybrid KEM [37,19], and a hybrid signature [20], the same does not hold true for PAKEs. In fact, composing PAKEs in this way may lead to a worst-of-both-worlds scenario, where breaking either hardness assumption is enough to compromise the security of the PAKE.

Thirdly, in practice PAKEs may be deployed in a user-server setting in which the server must verify whether the user holds the correct password but it should not know the user's password in the clear, because knowing the password would allow the server to impersonate the user. A protocol that replaces the server's password with an expensive one-way hash of the password, a.k.a. the password *verifier*, is known as an asymmetric PAKE (aPAKE) [17]. There exist generic constructions of aPAKEs from signatures and zero-knowledge proofs [36,48] or key-hiding Authenticated Key Exchange [41,30], but currently post-quantum

instantiations of these building blocks are significantly more expensive than post-quantum KEM.[3]

Recent progress on post-quantum PAKEs. Recently Januzelli et al. [52] and Arriaga et al. [9], investigated the OEKE-style PAKE design first proposed by McQuoid et al. [65], and exhibited practical KEM-based PAKE constructions which are secure in the Random Oracle Model (ROM) and which can be instantiated with a post-quantum KEM like ML-KEM. This is an important development because there is currently no secure quantum analogue of the ideal cipher model as there is for the ROM [21].

Secondly, Hesse & Rosenberg [47] and Lyu & Liu [60] recently exhibited secure *PAKE combiner* constructions, which combine in a black-box way any two PAKE schemes which satisfy some additional properties (however, see more on this below) s.t. the resulting PAKE is secure as long as *either* of the underlying PAKE schemes is secure. The combiners of [47,60] are secure in the ROM, and if they can be applied to a PAKE based on classical hardness assumption and a PAKE based on post-quantum assumption, they would result in a practical secure hybrid PAKE.

Finally, Yanqi Gu [40] and Lyu et al. [61] recently showed generic aPAKE constructions from any PAKE and IND-CCA2 KEM. Since this PAKE(+KEM)-to-aPAKE compiler makes no further requirements on the KEM, and the proofs hold in the ROM, applying such compiler to a hybrid KEM and a hybrid PAKE results in a practical secure hybrid aPAKE.

Missing security proofs. While all the above protocols are secure in isolation, their security proofs do not securely compose, and one cannot conclude that their composition instantiates a secure hybrid PAKE or hybrid aPAKE. We briefly go over the technical reasons why the proofs are incompatible and what is missing.

The first problem originates in the two existing proofs for the PAKE combiner which uses sequential composition [47,60]. (Both of these works analyze also PAKE combiners which use parallel composition but these require stronger assumptions on the underlying PAKEs.) In the proof by Hesse & Rosenberg, the second PAKE must satisfy a (statistical) *pre-shared key equality-hiding* (PSK-EH) property. The PSK-EH property asks that an adversary cannot learn whether two interacting parties use matching inputs in the case these inputs are (equal or independent) strong cryptographic keys, and the hiding must be information-theoretic because the property must hold even if the hardness assumption necessary for the standard security of this PAKE is broken. The proof by Lyu & Liu effectively considers only PAKE executions performed on matching passwords, hence their proofs do not require that the second PAKE has the PSK-EH property, but at the same time their security claims do not extend to the general case where parties can input mistyped or misremembered passwords.

[3] There are also constructions of *strong* aPAKEs from Oblivious Pseudorandom Functions (OPRF) [53], but current post-quantum OPRF proposals are even more expensive, and we leave strong aPAKEs as beyond the scope of this report.

The PSK-EH property is satisfied by the 'full EKE' KEM-to-PAKE compilers, e.g. [13,52], which password-encrypt both the KEM public key and the KEM ciphertext. However, it is not straightforward to satisfy it by the '*O*EKE-style' PAKEs, e.g. [52,9], which password-encrypt only *one* protocol flow, namely the KEM public key. In fact, if the sequential PAKE combiner is instantiated with the OEKE-style PAKE [52,9] built from ML-KEM, the statistical PSK-EH property does *not* hold because the ciphertext is a valid sample of the decisional MLWE problem only if the inputs match.

Furthermore, the compilers of [40,61] do not consider a PAKE which realizes the so-called *lazy extraction* variant of PAKE functionality introduced by Abdalla et al. [1]. The above results on the sequential PAKE combiner [47,60] show that the hybrid PAKE instantiated with CPace would realize a (slightly relaxed variant of) lazy extraction UC PAKE notion. The current proofs for the PAKE-to-aPAKE compilers [40,61] are thus missing two aspects of the hybrid PAKE which results from sequential composition of CPace and an ML-KEM based PAKE [52,9], namely that in the case that the MLWE assumption breaks and the ML-KEM based PAKE is insecure such PAKE would be (i) password-equality revealing, and (ii) it would be a lazy extraction PAKE and not a standard PAKE.

Our contributions. In this report we specify three protocols which are variants of the above constructions, including a post-quantum PAKE, a hybrid PAKE, and a hybrid aPAKE. Our protocols differ from the proposals above in some details which we explain below (and further in Section 3), but most differences can be seen as specific choices of component tools, motivated by practical considerations including ease of implementation. The three protocols we specify charter a complete path from a post-quantum KEM, specifically ML-KEM, and a PAKE based on classical assumptions, specifically CPace, to a hybrid PAKE and a hybrid aPAKE. Finally, we supply the security proofs which were left out in the above-cited works and which show that these protocols indeed compose as claimed. The three protocols we propose are as follows:

1. *OQUAKE*, a variant of protocol *NoIC* shown by Arriaga et al. [9]. (The 'O' stands for *one* password encryption and 'QU' for quantum-security.) Protocol *NoIC* [9] is an OEKE-style compiler which builds UC PAKE from a KEM. It needs the KEM to be one-way secure (OW) and anonymous (ANO) under the Plaintext Checking Attack (PCA), and to have (pseudo)uniform public keys (UNI) i.e. public keys must be indistinguishable from random group elements. ML-KEM satisfies these properties under MLWE since it is both IND-CCA [22] and ANO-CCA [39,76,64] under MLWE. Consequently, OQUAKE instantiated with ML-KEM yields UC PAKE secure in the ROM under the same assumption that underlies ML-KEM, which is the standardized post-quantum KEM [67]. Section 3.1 describes OQUAKE and the differences from NoIC in more detail.
2. *CPaceOQUAKE*, a variant of the sequential PAKE combiner of Hesse & Rosenberg [47], instantiated with CPace [31] as PAKE #1 and OQUAKE as PAKE #2. Section 3.2 describes CPaceOQUAKE in more detail.

3. *CPaceOQUAKE+*, a variant of the PAKE(+KEM)-to-aPAKE compilers of [40,61]. The protocol we present is a black-box compiler from PAKE to aPAKE, which additionally requires an IND-CCA secure KEM, similarly as in [40,61]. When the PAKE is instantiated with CPaceOQUAKE and the KEM is a hybrid KEM [28], we call such instantiation of this compiler CPaceOQUAKE+. Section 3.3 describes CPaceOQUAKE+ in more detail.

There are several key properties that we took into account when designing these protocols. Specifically, we set out to:

- Minimize the number of distinct primitives involved: We use only a KEM for public key operations; there is no need for signatures.
- Use standardized primitives or ones that have a clear path to standardization.
- Keep the bandwidth cost in the order of kilobytes.
- Keep the computational cost comparable to classical PAKEs.
- Achieve the strongest form of post-quantum hybrid security.

Lastly, we supply the missing proofs which are necessary to ensure that the above protocols are secure. As with all the prior proofs for these constructions, all our proofs are in the Universal Composability (UC) framework. We note that most of our work on the protocols and security proofs was done independently of the encoding work of [43], the KEM-based PAKEs of [52,9], the PAKE combiners of [47,60], and the PAKE-to-aPAKE compilers of [40,61], which builds an increased confidence in the security of these protocols. However, since many of our proofs duplicate the ones above, we do not include all of them in this report.

We summarize our contributions regarding the security arguments for the above suite of protocols as follows:

i *Sequential PAKE combiner without PSK-EH:* The proof of Hesse & Rosenberg [47] for a sequential hybrid PAKE does not work for CPaceOQUAKE because OQUAKE using ML-KEM does not have the PSK-EH property, i.e. it does not statistically hide input equality on full-entropy inputs. The proof by Lyu & Liu [60] is likewise incomplete because it effectively addresses the case of matching password inputs.

ii *KEM-based PAKE-to-aPAKE for RPE-PCS relaxed lazy-extraction PAKEs:* We provide a proof for the security of the CPaceOQUAKE+ protocol s.t. it applies to weaker underlying PAKEs. Specifically, our proof shows that the protocol transforms (RPE-PCS) relaxed lazy-extraction PAKE [1] into (RPE-PCS) relaxed aPAKE.

iii To enable the above we also propose new ideal functionalities for PAKEs that combine the (relaxed) lazy extraction introduced by Abdalla et al. [1] with an interface that allows leakage of password equality. (We note that a weaker version of the latter property was part of the PAKE model proposed by Shoup [74]). Our model also strengthens prior models for asymmetric PAKE (aPAKE) by supporting interactive initialization and by not treating user account or registration information as public.

Outline. In Section 2, we provide an in-depth overview of techniques of instantiating a post-quantum PAKE, which motivates our choice for protocol OQUAKE. Next, in Section 3, we present the three protocol proposals, OQUAKE, CPaceOQUAKE, and CPaceOQUAKE+. In Sections 4 and 5 we first define the ideal functionalities we aim to realize, and then provide proof sketches that highlight how the proposed protocols realize these ideal functionalities. Finally, we conclude in Section 6. The full specifications of the functionalities are in Appndix A while the full details of the proofs are in Appendices B, C, and D.

2 Existing constructions

In this section, we provide an overview of related work with respect to symmetric PAKEs and asymmetric PAKEs that are instantiable with post-quantum primitives. We start by examining what post-quantum instantiations currently exist for several primitives relevant to PAKEs including for uniform KEMs. We proceed with discussing symmetric and asymmetric PAKEs. We stress that, at the time of writing, we are not aware of any proposals for hybrid uniform KEMs or for hybrid (a)PAKEs.

2.1 Post-quantum primitives

We briefly discuss primitives that are relevant for instantiating PAKEs, for which a post-quantum instantiation exists. While we give several examples of post-quantum instantiations, we do not try to give a comprehensive overview.

Uniform key exchanges (uKA). Some general constructions convert a two-round key exchange into a PAKE. These constructions typically require uniformity properties of the messages exchanged in this key exchange. For example, the first message may be required to be indistinguishable from a randomness. The same may apply to the second message, or it may be required that the second message does not reveal anything about the first message.

In the context of post-quantum cryptography, a typical two-round key exchange is realized by a key encapsulation mechanism. Uniform KEMs are key encapsulation mechanisms that provide IND-CCA2 security and satisfy some of the uniformity properties described above.

There are currently no standardized uniform KEMs. However, ML-KEM is standardized and can be amended to achieve uniformity. One such construction is Kemeleon, which was used in an obfuscated post-quantum key exchange [43]. The construction does not work for all public keys and ciphertexts, so it must use rejection sampling. Beguinet et al. [13] show that *uncompressed* ML-KEM is also a uniform KEM. Alternatively, FrodoKEM [7] is already uniform, but it is not standardized.

Besides KEMs, there are also key exchanges that follow more closely to a Diffie-Hellman based key exchange, such as those based on isogenies. However, it is typically hard to achieve uniformity and these approaches are not yet standardized.

Uniform non-interactive key exchange (uNIKE). Some works show how to specifically transform a *non-interactive* key exchange into a PAKE. Such a NIKE requires both parties to send a message to the other party that does not depend on the other message. These works [73,12] also require the NIKE to be uniform; the messages need to be computationally indistinguishable from a random sample from the message space. An example of a lattice-based NIKE is Swoosh, which is uniform [33].

Lossy public key encryption (LPKE). Lossy public key encryption was introduced by Lyu et al. [62], along with two post-quantum instantiations. An LPKE is a labeled public key encryption scheme in which key generation binds the public key to some label. It permits some function IsLossy that allows someone with the trapdoor to tell if a public key corresponds to the label. Their first construction is based on the learning-with-errors problem, while their second construction is based on group actions (e.g. based on isogenies).

Smooth projective hash function (SPHF). Projective hash functions are hash functions that can be computed in two ways on inputs from a specific language. For example, the language of valid encryptions of a certain message for a given encryption scheme. The first way to compute a hash of an element in the language is using a secret hashing key for that given language. The second way is to use a public projection key and a witness of the fact that this element is indeed in the given language. The smoothness property requires that a computed hash is still close to uniform when the hash function is queried on elements that are not in the language.

Smooth projective hash functions were proposed as a tool for creating efficient CCA-secure public key encryption in the standard model. In the same way, while other PAKEs are only proven secure in the ROM/ICM, these SPHF-based PAKEs are proven secure in the standard model.

There are several post-quantum instantiations for lattice-based SPHFs. The first was proposed by Katz & Vaikuntanathan [57]. Benhamouda et al [18] later provide a more efficient SPHF in the standard model, and Zhang & Yu [79] provide an alternative in the random oracle model. We are not aware of any standardized SPHFs.

Oblivious transfer (OT). An oblivious transfer is a two-party functionality that allows a receiver to choose one of two messages that it will receive from a sender. The sender does not learn which message was chosen, and the receiver does not learn the other message. There are lattice-based constructions under the learning-with-errors assumption [71], and ones based on group actions (e.g. based on isogenies) [6].

2.2 Symmetric PAKEs

We provide a comprehensive overview of PAKEs with post-quantum instantiations at the time of writing. Table 1 shows a summary of all these works. We organize the PAKEs in this subsection by blueprint, but we note that these

blueprints are generalizations and may not be secure in all instances. Where possible, we ignore versions of the protocols with added key confirmation flows, focusing on the part of the protocols that differs.

Table 1: Potentially post-quantum PAKEs. We use * to indicate that the number of rounds is lower if both parties can initiate the protocol at the same time.

PAKE				**Properties**			**Security proof**					
Construction		*Protocol*		*Eff.*	*Auth.*		*Proof*		*Assumptions*			*Corr.*
Blueprint	Primitive	Name	Year	Rnd	U	S	Ref.	Func.	Model	CRS	Eras.	Adapt.
EKE	RLWE	RLWE-PPK	2016	2	○	○	[29]	BPR	ROM	○	○	○
	uKA	sPAKE	2020	2	○	○	[65]	$\mathcal{F}_{\mathsf{pwKE}}$	ROM	○	○	○
		CAKE	2023	2	○	○	[13]	$\mathcal{F}_{\mathsf{pwKE}}$	ICM	○	●	●
		·	·	·	·	·	[69]	BPR	ICM	○	○	○
	uNIKE	EKE-KA	2023	2*	○	○	[73]	$\mathcal{F}_{\mathsf{pwKE}}$	ICM	○	○	○
		EKE-NIKE	2024	2*	○	○	[12]	$\mathcal{F}_{\mathsf{bPAKE}}$	ICM	●	◐	◐
OEKE	PKE*	PAPKE-2-PAKE	2019	2	○	○	[23]	$\mathcal{F}_{\mathsf{pwKE}}$	ICM	○	○	○
	uKA	EKE-KEM	2023	2	○	●	[73]	$\mathcal{F}_{\mathsf{pwKE}}$	ICM	○	○	○
		OCAKE	2023	2	○	●	[13]	$\mathcal{F}_{\text{PWKE-SA}}$	ICM	○	○	○
		·	·	·	·	·	[8]	BPR	ICM	○	○	○
		CHIC	2024	2	○	●	[10]	$\mathcal{F}_{\mathsf{pwKE}}$	ICM	○	●	○
		OQUAKE	2025	2	○	●	3.1	$\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{LE}}}$	ROM	○	○	○
	LPKE	$\mathsf{PAKE}^{\mathsf{RO}}$	2024	3	●	●	[62]	$\mathcal{F}^*_{\mathsf{PAKE}}$	ROM	●	○	○
		$\mathsf{PAKE}^{\mathsf{QRO}}$	2024	3	●	●	[62]	$\mathcal{F}^*_{\mathsf{PAKE}}$	QROM	●	○	○
GL/KV	SPHF	F-PaKE	2003	3	●	○	[35]	BPR	Std.	●	○	○
		KV-3R	2009	3	○	○	[57]	BPR	Std.	?	?	?
		KV-1R	2013	2*	○	○	[58]	BPR	Std.	●	○	○
		KV-UC	2013	2*	○	○	[58]	$\mathcal{F}_{\mathsf{pwKE}}$	Std.	●	○	○
		GL-PAKE	2015	2	○	○	[2]	BPR	Std.	●	○	●
		KV-PAKE	2015	2*	○	○	[2]	BPR	Std.	●	○	●
	OT	$\mathsf{s}\Pi_{\mathsf{REfromOT}}$	2012	c	●	●	[25]	$\mathsf{s}\mathcal{F}_{\mathsf{re}}$	Std.	●	○	●
GK	SPHF	GK-3R	2010	3	●	●	[38]	BPR	Std.	●	○	○
		GK-UC	2010	4	●	●	[38]	$\mathcal{F}_{\mathsf{pwKE}}$	Std.	●	○	○
		GK-PAKE	2015	2	○	●	[2]	BPR	Std.	●	○	●
	OT	Conc. PAKE	2012	3	●	●	[25]	$\mathcal{F}_{\mathsf{pwKE}}$*	Std.	●	○	○

EKE-style protocols. Bellovin & Merritt proposed the first PAKE [17], which is called the encrypted key exchange (EKE). While the generic construction turns out not to always realize a secure PAKE [70], it has been used as a blueprint for many follow-up works. The key idea is to encrypt the flows of a key exchange using the password as a key. The construction requires two rounds if implicit

authentication is sufficient. One can add key confirmation flows resulting in a three-round PAKE with mutual authentication. A major challenge in instantiating post-quantum PAKEs derived from EKE is in instantiating the cipher, which is often assumed to be an ideal cipher.

RLWE-PPK Ding et al. [29] propose a PAKE based on the ring learning-with-errors problem, which is a modification of the authenticated key exchange presented by Zhang et al. [80]. We interpret this scheme as following the EKE blueprint because it essentially encrypts the flows of an authenticated key exchange, where the encryption mechanism is that of a one-time pad (using a hash of the password). Gao et al. [34] propose parameters for this protocol such that it can use the number-theoretic transform, making it more computationally efficient. Yang et al. [77] provide further optimizations, and Ren [72] extend it to the module learning-with-errors problem. We note that while this work claims to achieve an asymmetric PAKE, it does not fit the requirements: if the verifier is leaked from the server, anyone can impersonate the user as they do not need to know the password.

sPAKE McQuiod et al. [65] propose a new primitive called programmable-once public functions (POPFs), which are publicly-evaluatable functions that can be programmed (i.e. hardcoded) on exactly one input. The authors show that these POPFs can take the place of the cipher in an EKE construction. Their instantiation is essentially a two-round Luby-Rackoff construction, which they prove secure in the random oracle model. The security of this construction has been drawn into question. Freitas et al. [73] argue that the POPF is potentially malleable, which raises the question whether this instantiation of EKE can be proven secure in the universal composability framework. A new analysis by Januzelli et al. [52] addresses these issues for both EKE PAKEs and their one-encryption variants (OEKE). This new work provides a UC proof in the ROM for both approaches.

CAKE A PAKE based on a uniform KEM that follows the EKE blueprint, called CAKE [13]. For a cipher, it relies on the ideal cipher model. Beguinet et al. [13] provide a proof of security in the universal composability framework, while Pan & Zeng [69] provide a tighter proof in the BPR model[14] (see Section 4.2 for more details about this model). Pan & Zeng make two notes about the UC proof for CAKE. Firstly, they draw into question the security against replay attacks (of the first message). Secondly, they require a stronger form of ciphertext uniformity that remains secure when the adversary has access to a plaintext-checking oracle that answers if a certain key is a correct guess for a certain ciphertext.

EKE-KA Freitas et al. [73] propose an alternative to POPFs (see above) called half-ideal ciphers, which still rely on ideal ciphers, except that these ciphers are over bitstrings. The authors provide two protocols. One is called EKE, but we refer to it as EKE-KA. This schemes relies on a uniform non-interactive key exchange. It is proven secure in the UC framework.

EKE-NIKE Another scheme relying on uniform NIKEs has been proposed by Barbosa et al. [12]. The authors also prove that this PAKE is secure in the UC framework, and argue how to extend the proof to withstand adaptive corruption.

OEKE-style protocols. It is possible to adapt the EKE blueprint, which achieves implicit authentication, to achieve explicit authentication for the server in two flows. This new blueprint, called one-encryption EKE, or OEKE [24], does not encrypt the second flow. Instead, the server returns an explicit authentication tag.

PAPKE-2-PAKE Bradley et al. [23] propose password-authenticated public-key encryption (PAPKE) schemes. These are labeled PKEs (much like LPKE described above), in which the key generation and encryption algorithms also input a password. The decryption algorithm returns whether the ciphertext was created with the same ciphertext as the public key. The authors show that PAPKE implies a PAKE in the ideal cipher model, and they also show how to create a PAPKE from a uniform PKE (analogous to a uniform KEM).

OCAKE OCAKE is the OEKE version of CAKE [13]. The authors provide a UC proof in the ideal cipher model, and a later work by Alnahawi et al. [8] provides a tighter proof in the BPR model, who showed that the anonymity property in the original work is too strong. The ideal functionality $\mathcal{F}_{\text{PWKE-sA}}$ is a stronger version of $\mathcal{F}_{\mathsf{pwKE}}$, in which the server is explicitly authenticated.

EKE-KEM Freitas et al. [73] propose another protocol called EKE-KEM that uses the half-ideal cipher for encryption. The proof is in the universal composability framework.

CHIC Arriaga et al. [10] propose a special version of the half-ideal cipher that is more compact. The protocol still uses the OEKE blueprint, and is therefore similar to EKE-KEM. The proof, which is given in the universal composability framework, requires a stronger version of ciphertext uniformity, in which the adversary has access to a plaintext-checking oracle.

LPKE-based PAKE Lyu et al. [62] propose a PAKE based on lossy public key encryption (LPKE), as defined above. The authors provide two realizations of LPKE, both conjectured to withstand quantum attacks. We argue that this protocol roughly follows the OEKE blueprint when expanding the lattice-based realization of LPKE, in which the cipher becomes a one-time pad. The UC proof for this protocol is in the ROM, and with respect to a reformulation of Shoup's ideal functionality, which is in turn based on the original PAKE ideal functionality by Canetti et al (see Section 4.2). We denote it by $\mathcal{F}^*_{\mathsf{PAKE}}$.

GL/KV-style protocols. The protocols by Gennaro & Lindell [35] and Katz & Vaikuntanathan [57] do not follow the EKE or OEKE blueprint. Instead, in these protocols, the parties send each other a commitment of the password and a projection key for an SPHF.The language of this projection key is that of valid

commitments for each party's password. Each party can use the other's projection key and its own commitment to generate a hash, which will match with the hash produced using the hashing key if the passwords match. This provides implicit authentication. We consider a slight generalization of this blueprint that is not bound to SPHFs. We describe the steps in this blueprint as follows:

- Both parties provide the other with a means of obtaining a value indexed by a password, such that they know the value belonging to their password.
- Both parties query using their password.
- Both parties derive a session key based on the value they obtained and the value they know.

It is often indeed necessary for both parties to perform such a query to prevent a malicious party from determining the session key. Notice that all the protocols in this category are proven secure in the standard model.

F-PaKE One of the first PAKEs proven secure in the standard model is that by Katz et al. [56]. However, their protocol is strictly proven under the DDH assumption. Gennaro & Lindell [35] provide an abstraction of this PAKE using SPHFs and prove its security in the BPR model.

KV-3round - KV-3R Katz & Vaikuntanathan [57] propose another SPHF-based PAKE. The authors refer to it as a 3-round PAKE, so we refer to it as KV-3R. While there is a start of a security proof, the proof is given in a full version of the work that we were unable to access.

KV-1round - KV-1R Katz & Vaikuntanathan [58] also propose a single-round SPHF-based PAKE. However, in our overview, we consider the user to make the first request, so in practice it will result in a two-round PAKE. We refer to it as KV-1R.

KV-1round - KV-UC Finally, Katz & Vaikuntanathan [58] propose a variant of their single-round PAKE that is secure in the universal composability framework. We refer to this protocol as KV-UC. This protocol is slightly different from the protocol that was proven secure in the BPR model, because the BPR model is much weaker with regard to correct password guesses: When a correct password guess is made, the simulation stops, but in the UC framework the simulation continues.

GL-PAKE Abdalla et al. [2] propose a revised version of F-PaKE that requires only two rounds. In this protocol, both parties send each other a projection key for the SPHF, and there is no third flow in which a signature is sent.

KV-PAKE Abdalla et al.[2] also propose a revised version of KV-1R. Compared to this protocol, the parties do not send a hashing key hash but the projection key of the SPHF. Moreover, it uses an IND-PCA PKE scheme instead of IND-CCA.

$s\Pi_{\mathit{REfromOT}}$ Canetti et al. [25] propose multiple PAKEs based on oblivious transfers. The idea here is for each party to choose 1-out-of-D random strings, where each party has control over these random strings. The PAKE realizes $\mathsf{s}\mathcal{F}_{\mathsf{re}}$, where $\mathcal{F}_{\mathsf{re}}$ is essentially a stronger variant of $\mathcal{F}_{\mathsf{pwKE}}$ that provides mutual authentication. I.e. if both parties contribute the same input, the output is a random shared key. Otherwise, the output is empty (or $\perp$). The 's' denotes that this functionality is split following the transformation described by Barak et al. [11].

GK-style protocols. In the same way that OEKE is a modification of EKE with explicit server authentication, GK-style protocols [38] can be seen as a modification of GL/KV-style protocols. In these protocols, the user only commits to a password, while the server enables the user to query randomness using its password. We consider a generalization of this blueprint with the following steps:

- The server provides the user with a means of obtaining a value indexed by a password, such that the server knows the value belonging to their own password.
- The server sends an authentication tag of the value belonging to their password to the user.
- The user queries using their password and checks the authentication tag.
- Both parties derive a session key based on the value they obtained.

All the protocols in this category are proven secure in the standard model.

GK-3R Groce & Katz [38] propose an abstraction and generalization of the PAKE proposed by Jiang & Gong [55]. In this protocol, it is only the server that generates a projection key for the user to query. The authentication tag is an encryption of the password, where the randomness is the hash generated using the hashing key.

GK-UC Groce & Katz [38] also propose a version of the PAKE that is secure in the universal composability framework. In this version, both parties first exchange an encryption of the password, and the server includes a zero-knowledge proof that its second message encrypts the same password as the first.

GK-PAKE Abdalla et al. [2] simplify the two protocols above using a different kind of encryption scheme. In this PAKE, the user can tell if the server's password does not match, but the server does not learn if the user has the same password.

Concurrent PAKE Canetti et al. [25] propose a 'concurrent PAKE' based on the general GK blueprint. This scheme is based on the security of a weaker variant of oblivious transfers. The authors present a realization of this weaker OT using lattice-based cryptography.

2.3 Asymmetric PAKEs

Asymmetric (or augmented) PAKEs do not require the server to remember the user's password, which would allow it to impersonate the user. Instead, the server stores a verifier, from which it is impractical to derive the user's password or impersonate the user. We are not aware of any post-quantum aPAKEs, but we are aware of several methods for transforming a symmetric PAKE to an asymmetric one. We provide a brief overview of these methods based on the work by Gu [40], see Table 2 for a summary. We are also unaware of hybrid aPAKEs, and the methods described here would all rely on a hybrid PAKE in order to realize a hybrid aPAKE.

Table 2: An overview of PAKE-to-aPAKE transformations. The number of rounds reflects the transformation applied to OQUAKE (see Protocol 1).

Method	Ref.	Rounds	Primitive	Security model	Ideal functionality
Ω-method	[36]	3	Signature	ROM	$\mathcal{F}_{\mathsf{apwKE}}$
Hwang et al. 1	[49]	3	NIZK	ROM	$\mathcal{F}_{\mathsf{apwKE}}$
Hwang et al. 2	[49]	3	DDH	ROM	$\mathcal{F}_{\mathsf{apwKE}}$
Lyu et al.	[61]	3	KEM + AE	ROM	$\mathcal{F}^*_{\mathsf{apwKE}}$ with EA
APAKEM	[40]	3	KEM + AE	ROM	$\mathcal{F}_{\mathsf{apwKE}}$ with EA
Our method	3.3	3	KEM	ROM	$\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$

Ω-method Gentry et al. [36] propose the Ω-method and an ideal functionality for asymmetric PAKEs. While previous methods exist, such as the Z-method [63], these works did not include a formal security proof. Gentry et al. show that the Z-method does not realize a secure aPAKE. The method works by first establishing a confidential channel using the verifier of the password, after which the user proves knowledge of the password by signing the protocol transcript. The user obtains the signing key from the server, which stores an encryption of the signing key that can be decrypted using the password.

Hwang et al. Hwang et al. [49] propose two methods for realizing an aPAKE using a PAKE. Their first method is based on the work by Gentry et al. [36], using a non-interactive zero-knowledge proof to prove that the user knows the password. Their second method uses a Diffie-Hellman KEM. The user proves that it knows the password by showing that it knows the corresponding KEM secret key. It does so by succesfully decapsulating a fresh KEM ciphertext and incorporating the key in a key confirmation tag.

Lyu et al. The construction by Lyu et al. [61] is a generalization of the second method by Hwang et al. [49]. The KEM is no longer required to be based on a Diffie-Hellman assumption. The only assumption is that the KEM achieves

IND-CCA2 security. In this aPAKE, the parties first establish a confidential channel using a PAKE and authenticated encryption (AE). They then perform a challenge-response with the KEM and a message authentication code, achieving explicit authentication (EA). The authors prove security with respect to a modified version of $\mathcal{F}_{\mathsf{apwKE}}$ including some of the changes proposed by Shoup [74]. As a result, it does not require pre-established session identifiers. [4]

APAKEM In parallel to the construction by Lyu et al., Gu designed an almost identical construction called APAKEM [40]. While there are some differences in the hashes that are derived, the largest difference is in the fact that APAKEM is proven with respect to the classic aPAKE functionality that does not incorporate Shoup's modifications [74]. In other words, it requires a pre-established session identifier.

Our method In Section 3.3 we propose a method that is very similar to above two constructions with a few slight modifications. Firstly, our protocol does not use authenticated encryption (it uses a one-time pad) or a MAC (it uses key confirmation tags derived using the random oracle). Secondly, the session keys derived in our protocol use the challenge KEM ciphertext's key as key material, so that a bad implementation that does not (correctly) verify the challenge response cannot output the correct session key. Most importantly, we prove security of this aPAKE with respect to a different ideal functionality. The other methods prove security with respect to (variants of) $\mathcal{F}_{\mathsf{apwKE}}$, but Hesse [45] highlights some issues with this functionality. The ideal functionality that we use ($\mathcal{F}^{*}_{\mathsf{aPAKE\text{-}EA^R}}$) is a variation of Shoup's ideal functionality [74]. Our proof also works for a more general class of PAKEs, namely those that may leak password equality. The PAKE may also use lazy extraction.

3 The CPaceOQUAKE+ protocol suite

Upgrading from a classical to a post-quantum PAKE would ideally strictly increase security, so that the PAKE resists all the attacks it did previously as well as those launched by a quantum-capable attacker. In other words, the PAKE must by hybrid: it must remain as secure as classical PAKEs under classical assumptions, while resisting quantum attacks by relying on newer, post-quantum assumptions. Next to that, we want to achieve an asymmetric PAKE, so that one of the parties (the server) does not know the password, but only a verifier. While previous work [52,9,47,60,40,61] provides building blocks to achieve such a hybrid aPAKE, there are several small gaps in the security proofs that make these building blocks incompatible to compose. In this section, we propose CPaceOQUAKE+, which combines these building blocks to construct a hybrid aPAKE, while addressing the missing gaps in the security proofs.

[4] One might still use a session identifier to bind to an existing session in a higher level protocol, or one may consider using the transcript of the protocol so far.

3.1 OQUAKE: A post-quantum PAKE

We first discuss a post-quantum PAKE called OQUAKE, presented as Protocol 1. OQUAKE is a version of the NoIC protocol proposed by Arriaga et al. [9]. NoIC distinguishes itself from other OEKE-style PAKEs in that it does not require ideal ciphers in its security proofs. Instead, Arriaga et al. [9] and Januzelli et al. [52], following the PAKE design first proposed by McQuoid et al. [65], replace ideal ciphers with 2 rounds of Feistel (see also sPAKE under Section 2.2), constructing secure UC PAKE from KEM in the the random oracle model. This is an important property, because whereas extractability in the quantum analogue of the random oracle is well studied [78,27], the same does not yet hold for the ideal cipher model [75]. Moreover, it is not clear how to instantiate a quantum-secure ideal cipher: it has been shown that classically-secure constructions for strong PRPs such as a four-round Luby-Rackoff cipher [59] are not necessarily secure against quantum attackers [51].

Protocol NoIC relies on KEM which is one-way under plaintext-checking attack (OW-PCA), anonymous under PCA (ANO-PCA), and has uniform public keys [9]. Recall that the recently standardized ML-KEM [67] has all these properties under the M-LWE assumption: it has uniform public keys and it is IND-CCA [22] and ANO-CCA [39,76,64] under M-LWE, which implies OW-PCA and ANO-PCA. OQUAKE is an instantiation of NoIC that uses a public key encoding $\phi : \mathcal{P} \mapsto \{0,1\}^{\lambda_{pk}}$ which encodes a random ML-KEM public key in domain $\mathcal{P}$ as a random bitstring of length λ_{pk}[5]. This ensures that both hash functions in the 2 rounds of Feistel in NoIC [9] output bitstrings, and the group operations become XOR operations. The protocol differs in a few ways from the protocol described by Januzelli et al. [52], e.g. in the latter protocol the interaction transcript and the password are not included as inputs in the key+authenticator derivation hash $\mathtt{H}_{\mathtt{kc}}$. We note that including the password in this hash removes the collision-resistance requirement on the KEM present in [52], and it allows use of ML-KEM, which is slightly different from Kyber [22].

3.2 CPaceOQUAKE: A hybrid PAKE

We present a hybrid PAKE protocol CPaceOQUAKE, shown in Protocol 3. It is a sequential composition of PAKE protocol CPace, which bases its security on a variant of the gap Diffie-Hellman (GDH) assumption [4], and OQUAKE. The composition is almost identical to the sequential PAKE combiner construction proposed by Hesse & Rosenberg [47], and it is also similar to the one proposed by Lyu & Liu [60].

One might wonder why we do not use a parallel composition to realize the hybrid PAKE, as it would reduce the number of interactions. The reason is that this requires both PAKEs to unconditionally hide the password. Specifically, it means that even if the underlying hardness assumption were to break, an adversary should not be able to extract the password from the PAKE interaction. The

[5] ϕ is the *Kemeleon* encoding proposed by Günther et al. [43]

Protocol 1: OQUAKE: variant of protocol *NoIC* [9], changes marked in gray

κ is a security parameter, λ_{pk} is an integer s.t. $\lambda_{pk} \geq \kappa$
KEM is a KEM and $\mathcal{P}$ is a set s.t. $\mathsf{KEM}.\mathrm{KeyGen}(1^\kappa)$ generates (pk, sk) s.t. $pk \in \mathcal{P}$
$\phi : \mathcal{P} \mapsto \{0,1\}^{\lambda_{pk}}$ is a uniform encoding of random $\mathcal{P}$ elements as random λ_{pk}-bit strings
$\mathtt{H}, \mathtt{H}', \mathtt{H}_{\mathsf{kc}}$ are hash functions onto resp. $\{0,1\}^{\lambda_{pk}}$, $\{0,1\}^{3\kappa}$, and $\{0,1\}^{2\kappa} \times \{0,1\}^{\kappa}$

User U on input $(\mathsf{ssid}, S, \mathsf{pw}_U)$		**Server** S on input $(\mathsf{ssid}, U, \mathsf{pw}_S)$
	$\mathsf{fsid} = (\mathsf{ssid}, U, S)$	
$(pk, sk) \leftarrow \mathsf{KEM}.\mathrm{KeyGen}(1^\kappa)$		
$bpk \leftarrow \phi(pk)$		
$r \leftarrow_R \{0,1\}^{3\kappa}$		
$T \leftarrow bpk \oplus \mathtt{H}(\mathsf{fsid}, \mathsf{pw}_U, r)$		
$s \leftarrow r \oplus \mathtt{H}'(\mathsf{fsid}, \mathsf{pw}_U, T)$		
$\mathbf{Epk} \leftarrow (s, T)$	$\xrightarrow{\mathbf{Epk}}$	
		$(s, T) \leftarrow \mathbf{Epk}$
		$r \leftarrow s \oplus \mathtt{H}'(\mathsf{fsid}, \mathsf{pw}_S, T)$
		$bpk \leftarrow T \oplus \mathtt{H}(\mathsf{fsid}, \mathsf{pw}_S, r)$
		$pk \leftarrow \phi^{-1}(bpk)$
		$(c, k) \leftarrow \mathsf{KEM}.\mathrm{Encaps}(pk)$
	$\xleftarrow{c, h}$	$(h, \mathsf{K}) \leftarrow \mathtt{H}_{\mathsf{kc}}(\mathsf{fsid}, \mathbf{Epk}, c, pk, \mathsf{pw}_S, k)$
$k \leftarrow \mathsf{KEM}.\mathrm{Decaps}(sk, c)$		
$(h', \mathsf{K}') \leftarrow \mathtt{H}_{\mathsf{kc}}(\mathsf{fsid}, \mathbf{Epk}, c, pk, \mathsf{pw}_U, k)$		
If $h' \neq h$ then re-set $\mathsf{K}' \leftarrow_R \mathbb{K}$		
$\mathsf{ftr} \leftarrow (\mathsf{fsid}, \mathbf{Epk}, c, h)$		$\mathsf{ftr} \leftarrow (\mathsf{fsid}, \mathbf{Epk}, c, h)$
Output $(\mathsf{K}', \mathsf{ftr})$		Output $(\mathsf{K}, \mathsf{ftr})$

only post-quantum PAKE that currently achieves this property is proposed by Günther et al. [42], which uses a modified version of FrodoKEM. Unfortunately, this KEM requires much larger ciphertext size, namely 144 kilobytes, and it is also not a part of the standardization process.

Classical PAKE. CPace [44], shown in Protocol 2, is a PAKE based on a variant of the Diffie-Hellman assumption, which was selected in the 2019 CFRG's PAKE selection process. CPace is a highly efficient scheme which can be instantiated over an elliptic curve that permits a hash-to-curve operation. CPace also has a 'quantum-annoying' property, which means that a quantum attacker must compute a discrete logarithm for each password guess. Eaton and Stebila [32] proved this property in the generic group model.

CPace is designed as a balanced PAKE, meaning that it is not presented as a user-server protocol but as a protocol between two arbitrary parties. However, in Protocol 2 we present protocol CPace in a way that fits the user-server scenario. Specifically, we fix the order of the party's identifiers U and S and we let the user initiate the protocol. For our security proofs, we rely on the proof by Abdalla et al. [4], which shows that CPace realizes functionality $\mathcal{F}_{\mathsf{lePAKE}}$ under a weaker

variant of the gap Diffie-Hellman problem, which roughly states that the computational Diffie-Hellman problem is hard even if one can solve the decisional Diffie-Hellman problem.

Protocol 2: Protocol CPace [44], with a fixed order of U, S identifiers.

$\mathbb{G}$ is a group of prime order p and I is an identity element in $\mathbb{G}$
$\mathtt{H}_{\mathbb{G}}$ is a hash function onto $\mathbb{G} \setminus \{I\}$

User U on input $(\mathsf{ssid}, S, \mathsf{pw}_U)$		**Server** S on input $(\mathsf{ssid}, U, \mathsf{pw}_S)$
	$\mathsf{fsid} = (\mathsf{ssid}, U, S)$	
$G \leftarrow \mathtt{H}_{\mathbb{G}}(\mathsf{fsid}, \mathsf{pw}_U)$		$G \leftarrow \mathtt{H}_{\mathbb{G}}(\mathsf{fsid}, \mathsf{pw}_S)$
$a \leftarrow_R \mathbb{Z}_p, \quad A \leftarrow aG$	$\xrightarrow{A}$	$b \leftarrow_R \mathbb{Z}_p, \quad B \leftarrow bG$
	$\xleftarrow{B}$	
$Z \leftarrow aB, \quad$ abort if $Z = I$		$Z \leftarrow bA, \quad$ abort if $Z = I$
$\mathsf{K} \leftarrow \mathtt{H}_{\mathsf{K}}(\mathsf{ssid}, A, B, Z)$		$\mathsf{K} \leftarrow \mathtt{H}_{\mathsf{K}}(\mathsf{ssid}, A, B, Z)$
$\mathsf{ftr} \leftarrow (\mathsf{fsid}, A, B)$		$\mathsf{ftr} \leftarrow (\mathsf{fsid}, A, B)$
Output $(\mathsf{K}, \mathsf{ftr})$		Output $(\mathsf{K}, \mathsf{ftr})$

Hybrid PAKE. The Hybrid PAKE scheme CPaceOQUAKE, shown in Protocol 3, sequentially executes CPace and OQUAKE, where the password input into OQUAKE incorporates the session keys established by CPace as additional key material. Protocol CPaceOQUAKE follows the sequential PAKE combiner proposed by Hesse & Rosenberg [47], with two small modifications highlighted in protocol 3. Since CPace returns the key before the server's first message, we can switch roles in the execution of OQUAKE and have the server initiate it, saving one protocol flow. The output of the hybrid PAKE is a key that is derived from the session keys established by both sub-PAKEs. In Section 5, we show that CPaceOQUAKE realizes $\mathcal{F}^{*}_{\mathsf{PAKE}^{\mathsf{RLE}}}$, a relaxed lazy-extraction PAKE, as long as either of the two PAKEs remains uncompromised.

The only essential change CPaceOQUAKE makes from the combiner of [47] is in adding the transcript of PAKE #1 into the inputs of the hash that derives the password input of PAKE #2. This is also the choice made by the version of the sequential PAKE combiner due to Lyu & Liu [60], and the rationale for it is to bind PAKE #2 input uniquely to the PAKE #1 instance even if the session identifier input of PAKE #1 fails to be unique. This change is very small and since the protocol transcript is public it is clear that the security properties proven for the sequential combiner by Hesse & Rosenberg [47] extend to CPaceOQUAKE.

Protocol 3: CPaceOQUAKE, variant of sequential combiner of [47] (changes marked in gray) instantiated with CPace (in blue) and OQUAKE (in violet).

$\kappa, \lambda_{pk}, \mathsf{KEM}, \phi, \mathtt{H}, \mathtt{H}', \mathtt{H}_{\mathsf{kc}}$ are as in OQUAKE, i.e. Protocol 1
$\mathbb{G}, p, I, \mathtt{H}_G$ are as in CPace, i.e. Protocol 2
KDF is a PRG, $\mathrm{KDF} : \{0,1\}^{\kappa} \leftarrow \{0,1\}^{2\kappa}$, $\mathtt{H}_{\overline{\mathsf{pw}}}, \mathtt{H}_{\mathsf{out}}$ are hash functions onto range $\{0,1\}^{\kappa}$

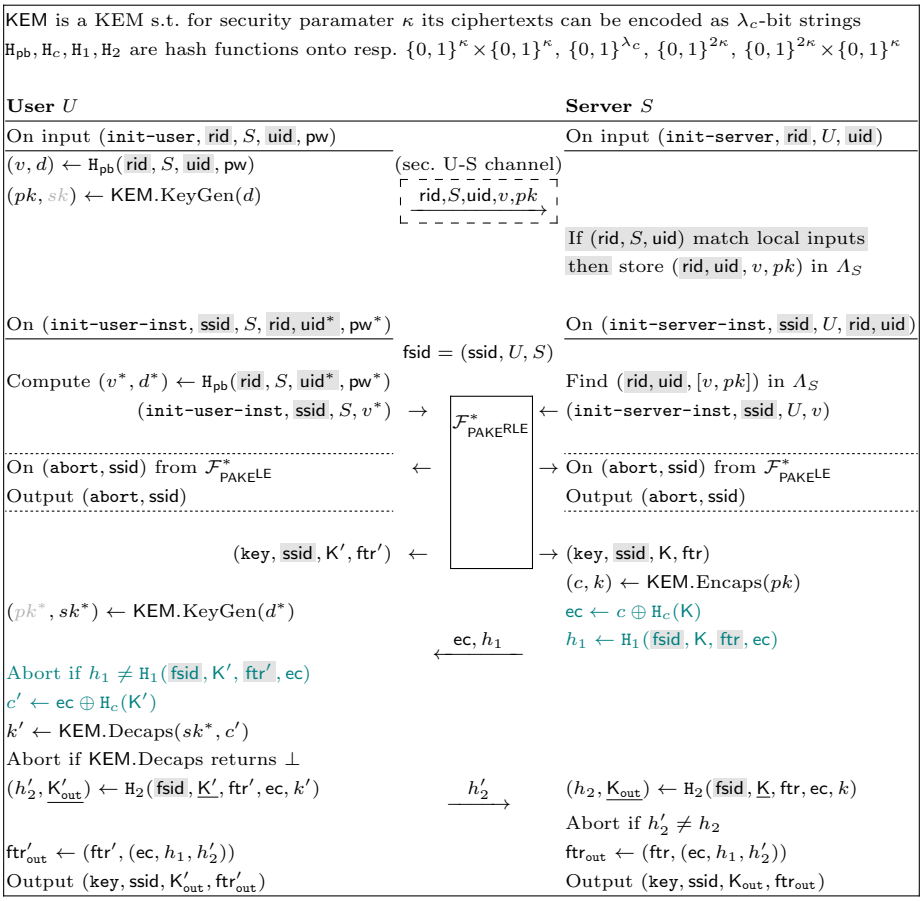
User U on input $(\mathsf{ssid}, S, \mathsf{pw}_U)$		**Server** S on input $(\mathsf{ssid}, U, \mathsf{pw}_S)$
	$\mathsf{fsid} = (\mathsf{ssid}, U, S)$	
$G \leftarrow \mathtt{H}_{\mathbb{G}}(\mathsf{fsid}, \mathsf{pw}_U)$		$G \leftarrow \mathtt{H}_{\mathbb{G}}(\mathsf{fsid}, \mathsf{pw}_S)$
$a \leftarrow_R \mathbb{Z}_p, \quad A \leftarrow aG$		$b \leftarrow_R \mathbb{Z}_p, \quad B \leftarrow bG$
	$\xrightarrow{A}$	
		$Z \leftarrow bA, \quad$ abort if $Z = I$
		$\mathsf{K}_1 \leftarrow \mathtt{H}_{\mathsf{K}}(\mathsf{ssid}, A, B, Z)$
		$\mathsf{K}_1^L \,\|\|\, \mathsf{K}_1^R \leftarrow \mathrm{KDF}(\mathsf{K}_1)$
		$\overline{\mathsf{pw}}_S \leftarrow \mathtt{H}_{\overline{\mathsf{pw}}}(\mathsf{fsid}, (A, B), \mathsf{pw}_S, \mathsf{K}_1^L)$
		$(pk, sk) \leftarrow \mathsf{KEM}.\mathrm{KeyGen}(1^{\kappa})$
		$bpk \leftarrow \phi(pk)$
		$r \leftarrow_R \{0,1\}^{3\kappa}$
		$T \leftarrow bpk \oplus \mathtt{H}(\mathsf{fsid}, \overline{\mathsf{pw}}_S, r)$
		$s \leftarrow r \oplus \mathtt{H}'(\mathsf{fsid}, \overline{\mathsf{pw}}_S, T)$
	$\xleftarrow{B \,\vdots\, \mathbf{Epk}}$	$\mathbf{Epk} \leftarrow (s, T)$
$Z \leftarrow aB, \quad$ abort if $Z = I$		
$\mathsf{K}_1 \leftarrow \mathtt{H}_{\mathsf{K}}(\mathsf{ssid}, A, B, Z)$		
$\mathsf{K}_1^L \,\|\|\, \mathsf{K}_1^R \leftarrow \mathrm{KDF}(\mathsf{K}_1)$		
$\overline{\mathsf{pw}}_U \leftarrow \mathtt{H}_{\overline{\mathsf{pw}}}(\mathsf{fsid}, (A, B), \mathsf{pw}_U, \mathsf{K}_1^L)$		
$(s, T) \leftarrow \mathbf{Epk}$		
$r \leftarrow s \oplus \mathtt{H}'(\mathsf{fsid}, \overline{\mathsf{pw}}_U, T)$		
$bpk \leftarrow T \oplus \mathtt{H}(\mathsf{fsid}, \overline{\mathsf{pw}}_U, r)$		
$pk \leftarrow \phi^{-1}(bpk)$		
$(c, k) \leftarrow \mathsf{KEM}.\mathrm{Encaps}(pk)$		
$(h, \mathsf{K}_2) \leftarrow \mathtt{H}_{\mathsf{kc}}(\mathsf{fsid}, \mathbf{Epk}, c, pk, \overline{\mathsf{pw}}_U, k)$	$\xrightarrow{c, h}$	
		$k \leftarrow \mathsf{KEM}.\mathrm{Decaps}(sk, c)$
		$(h', \mathsf{K}_2') \leftarrow \mathtt{H}_{\mathsf{kc}}(\mathsf{fsid}, \mathbf{Epk}, c, pk, \overline{\mathsf{pw}}_S, k)$
		If $h' \neq h$ then re-set $\mathsf{K}_2' \leftarrow_R \mathbb{K}$
$\mathsf{ftr} \leftarrow (\mathsf{fsid}, A, B, \mathbf{Epk}, c, h)$		$\mathsf{ftr} \leftarrow (\mathsf{fsid}, A, B, \mathbf{Epk}, c, h)$
$\mathsf{K} \leftarrow \mathtt{H}_{\mathsf{out}}(\mathsf{ftr}, \mathsf{K}_1^R, \mathsf{K}_2)$		$\mathsf{K} \leftarrow \mathtt{H}_{\mathsf{out}}(\mathsf{ftr}, \mathsf{K}_1^R, \mathsf{K}_2')$
Output $(\mathsf{K}, \mathsf{ftr})$		Output $(\mathsf{K}, \mathsf{ftr})$

3.3 CPaceOQUAKE+: A hybrid aPAKE

In an asymmetric PAKE, the server may not store the password. Instead, it stores a one-way function of the password, a.k.a. a *password verifier*, from which one can obtain the password only via a brute-force search, a.k.a. an *Offline Dictionary Attack*. An easy way to achieve an asymmetric PAKE from an existing PAKE is

to run the PAKE using the verifier instead of the password. However, this would allow any party to impersonate the user upon server compromise.

Protocol 4: Protocol CPaceOQUAKE+: Compiler from PAKE to asymmetric PAKE (aPAKE), which defines CPaceOQUAKE+ if (rle)PAKE is instantiated with CPaceOQUAKE. The compiler is a variant of [61], with changes marked in gray, plus in [61] the underlined input is the sole input in computing $\mathsf{K}_{\mathsf{out}}$. Text in teal is a specific instantiation of authenticated encryption.



Therefore, we need the user to prove that they have knowledge of the actual password, rather than the verifier. Previous work by Gentry et al. [66] shows how to do so using digital signatures. However, since we are already relying on the hardness of the M-LWE assumption and require a KEM in our PAKEs, we design a generic transformation from PAKEs to aPAKEs that relies on a KEM instead. In this section, we provide such a protocol that is in many ways similar

to the KEM-based transformations proposed by Gu [40] and Lyu et al. [61]. We describe related work in more detail in Section 2.3.

The key idea behind our protocol, presented in Protocol 4, is that while the PAKE's session keys do not allow us to establish an authenticated channel, the channel is confidential in the sense that it prevents eavesdropping from adversaries who do not know the verifier. This means that the KEM does not need to be anonymous: if the other party knows the verifier, then they already know as much as the server does.

In this protocol, the verifier is made up of two parts. The first part is a pseudo-random bitstring generated by a password-based random oracle $\mathtt{H}_{\mathtt{pb}}$, which in practice should be a slow oracle to query. This ensures that the server or an attacker that compromises the server must perform extensive computation to obtain the user's password. The second part is a KEM public key that is generated using a pseudo-random seed that is also generated by this oracle. The server holds onto this public key but it does not learn the seed. The security of the KEM implies, in particular, that it is computationally infeasible to find the seed given this public key or given the ciphertexts encrypted under this key.

The transformation we propose also achieves mutual key confirmation, and it executes as follows: The user hashes its password and account information using hash oracle $\mathtt{H}_{\mathtt{pb}}$ deriving the verifier and the seed from which the user derives the KEM secret key. The server retrieves the verifier and the KEM public key from the password file associated with the user's account. The two parties then run PAKE on the verifier, establishing a secure channel. Subsequently, the server encapsulates a KEM ciphertext from the public key and sends it to the user *over this secure channel*. This secure transmission is accomplished using a one-time pad and an RO-based MAC (these steps are highlighted in teal in protocol 4). The user then proves knowledge of the password by using the KEM secret key to decrypt the encapsulated key, and by creating an authentication tag which it sends to the server. The server then verifies this tag and aborts otherwise. The server-to-user authentication is established by the RO-based MAC in the above secure transmission. This MAC plays a role of an authentication tag in the server's first message, and the user aborts if this tag does not match.

To ensure that the steps taken after the inner PAKE finishes are bound to that execution, we include the full transcript ftr when we derive the hashes and the final key. This transcript includes all the messages that were exchanged in the PAKE. So, for CPaceOQUAKE+, this includes A & B from CPace, and **Epk**, c, and h from OQUAKE.

The differences between our PAKE-to-aPAKE compiler and the compilers of [40,61] involve additional inputs in the hash functions. Specifically, (1) we use an explicit long-term user identifier and a unique "account identifier", which can be implemented e.g. with a random salt, as additional inputs into the initial hash which derives the authenticator values, (2) we assume that each authentication attempt gets a unique (sub)session identifier, which we add to hash inputs, and (3) we include the full protocol transcript and the KEM-encapsulated key as inputs in the final key derivation hash. Moreover, we also (4) instantiate the

authenticated encryption block using a specific "one-time pad plus an RO-based MAC" implementation, and (5) we bind that authenticated encryption to the underlying PAKE by including its transcript in the MAC input. The changes are small and the proofs of [40,61] go through for our compiler, but we validate the protocol via an independent proof, which also adheres to a different (and in some aspects stronger) functionality specification (see more below).

4 Security definitions

A PAKE is a key exchange in which two parties establish a shared pseudo-random key if and only if they both enter the correct password. For the PAKE to be useful as a key exchange in some larger protocol, for example to establish a secure channel, we must prevent adversaries from learning anything about the key. Passwords are typically easy to brute force because they are only low-entropy secrets. For this reason, an adversary should not be allowed to perform an offline password guessing attack, in which the adversary learns whether a password guess is correct without having to interact with the involved parties. Furthermore, while each on-line interaction gives the adversary an ability to authenticate using some password guess, the protocol must ensure that the adversary can test at most one password in each authentication attempt.

In this section, we outline an ideal functionality that models this behavior. The ideal functionality we use includes the *lazy extraction* (LE) and *relaxed lazy extraction* (RLE) relaxations proposed by Abdalla et al. [1] and Hesse & Rosenberg [47], but adapted to the ideal PAKE conventions proposed by Shoup [74]. (In particular, we follow the latter by making some adversary choices more explicit than in the model proposed by Canetti et al. [26], see below.) However, one key difference from the functionalities of [1,47] is that we also include modifications, denoted resp. $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{LE}}}$ and $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$, which leak to the adversary whether two passively observed interacting parties hold matching passwords. Our formulation is also convenient for modeling *asymmetric* PAKEs with few changes.

The first ideal functionality for balanced PAKEs, which we denote $\mathcal{F}_{\mathsf{pwKE}}$, was proposed by Canetti et al. [26].[6] Subsequently multiple variants have been proposed that achieve slightly different notions of security, such as $\mathcal{F}_{\mathsf{lePAKE}}$ [1] and $\mathcal{F}_{\text{PWKE-SA}}$ [13]. Shoup [74] proposed an ideal functionality for *unbalanced* PAKEs, where the non-initiating party always uses the correct password, which is useful to model a user-server scenario, and this functionality also implicitly requires key confirmation. Shoup's functionality also leaked password-matching information but it did so irrespective of how the adversary acts in an interaction between these two parties, whereas we strengthen the model of [74] by allowing

[6] The original presentation of $\mathcal{F}_{\mathsf{pwKE}}$ in [26] suffers from an 'honest-party key corruption' problem which prevents it from composing into larger protocols (see [4], Appendix C). This mistake also slipped into several derived functionalities for augmented/asymmetric PAKEs [54,46,1]. We explain the problem in Section 4.2, and the simple adjustment in the functionality which fixes the problem, which is also incorporated in our PAKE functionality.

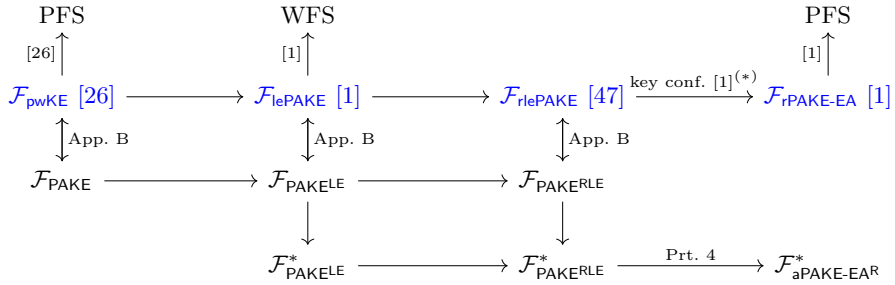
the adversary to learn the password-match predicate only on passively observed interaction. That is, our functionality allows the adversary to run the password-match testing query, but doing so disallows subsequent active attack on either session, although the adversary can still interrupt or abort them.

In this section we outline an ideal PAKE functionality which adopts a syntactic style of the functionality proposed by Shoup, but adapts it to a balanced PAKE with the adjustments outlined above. Our definitional framework allows expressing several variants of the PAKE functionality, but the default variant is denoted $\mathcal{F}^{*}_{\mathsf{PAKE}^{\mathsf{RLE}}}$, which incorporates a relaxed form of lazy extraction from [1,4].

We note that another important optional property of PAKE protocols is (explicit) *entity authentication* (EA), i.e. where the parties output a key only if their counterparty holds a matching password, and abort the protocol otherwise. (By contrast, in the standard notion of PAKE a party outputs a key in either case, but this key is pseudorandom to the adversary if the counterparty did not input a matching password.) Our models incorporate this optional feature, which we denote by adding '-EA' to the functionality designation.

We provide an overview of the relations between variants of the PAKE functionalities in Figure 1, where acronyms WFS and PFS denote respectively weak and perfect forward security property.

Fig. 1: An overview of the implications between PAKE functionalities. Functionalities marked in blue are from prior works. We connect functionalities A and B by an arrow without a label if B is a weakening of A. We note that our main target is $\mathcal{F}^{*}_{\mathsf{PAKE}^{\mathsf{RLE}}}$ for balanced PAKE and $\mathcal{F}^{*}_{\mathsf{aPAKE\text{-}EA}^{\mathsf{R}}}$ for asymmetric PAKE. (We mark the 'add key confirmation' transform $\mathcal{F}_{\mathsf{rlePAKE}} \to \mathcal{F}_{\mathsf{rPAKE\text{-}EA}}$ from [1] with an asteriks because it was shown for $\mathcal{F}_{\mathsf{lePAKE}} \to \mathcal{F}_{\mathsf{rPAKE\text{-}EA}}$, but the 'r' relaxation in the underlying PAKE does not affect the same relaxation in the outer PAKE.)



4.1 The universal composability framework

As explained by Shoup [74], there is no single definition of the universal composability framework. In this work, we will provide proofs that stick to the core of the UC framework, without taking the low-level specifics into account (similar

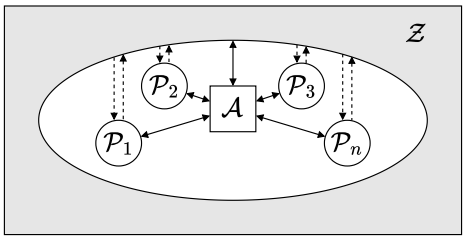


Fig. 2: The real world, where parties only communicate through the adversary $\mathcal{A}$, who has full control over the communication channels. The adversary reports to and is commanded by the environment $\mathcal{Z}$.

to many other works). In our proofs we focus on static adversaries, i.e. ones that corrupt parties prior to a protocol execution.

The high-level idea of UC security is that a protocol Π securely realizes an ideal functionality $\mathcal{F}$ if for every environment $\mathcal{Z}$ (acting in the role of a distinguisher) and adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$, where $\mathcal{Z}$ cannot tell whether it is interacting with a real-world execution of Π being manipulated by $\mathcal{A}$, or an ideal- world simulation provided by $\mathcal{S}$ interfacing with the functionality $\mathcal{F}$. In the real world, $\mathcal{F}$ interacts directly with the parties and the adversary, whereas in the ideal world it interacts with the parties and the simulator. This simulator $\mathcal{S}$ takes over the role of the adversary in the ideal world, and its objective is to closely match everything that is happening in the real world protocol, such as mimicking the messages that would otherwise be sent. If the simulation is indistinguishable from a real-world execution, this implies that the protocol does not leak more information than what is inherently leaked by $\mathcal{F}$, since $\mathcal{S}$ does not have access to secret information such as the parties' inputs and outputs.

Real world. In the real world, all parties $\mathcal{P}_1, \ldots, \mathcal{P}_n$ receive their inputs from the environment $\mathcal{Z}$, and proceed with the protocol. Communication never goes directly to another party; all messages are passed directly to the adversary, who therefore has full control of the communication channel, and can eventually decide to pass the messages on to their intended recipient. At the end of the protocol, the parties submit their output to $\mathcal{Z}$. An intuitive way to think about the environment is as an abstraction for anything else that could happen around this protocol. For example, this protocol could be run as part of multiple larger protocols. Fig. 2 shows an overview of the real world.

Ideal world. The ideal world represents the situation where there exists an oracle that performs exactly what we want to achieve (the ideal functionality $\mathcal{F}$), so parties do not have to resort to a complex protocol. Instead, the parties become dummies that defer their work entirely to $\mathcal{F}$: they directly provide it their inputs and eventually receive their outputs. To make this world indistinguishable from the real world, the adversary is referred to as the simulator $\mathcal{S}$, whose task

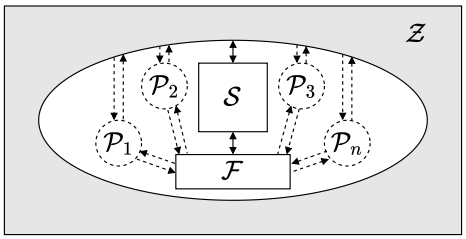


Fig. 3: The ideal world, where the parties defer their work to an ideal functionality $\mathcal{F}$, and the adversary is referred to as the simulator $\mathcal{S}$. Dashed lines represent communication lines that are only used to communicate inputs and outputs.

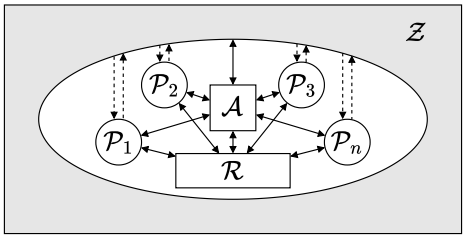


Fig. 4: A hybrid world, where the parties and the adversary have access to an oracle. Pictured here is the random oracle, denoted by $\mathcal{R}$.

is to use the $\mathcal{F}$ to simulate all that would otherwise happen in the real world. Fig. 3 shows an overview of the ideal world.

Hybrid world. The hybrid world is useful to describe real world protocols where certain primitives or sub-protocols have been abstracted away using an oracle. A common example is the random oracle model, in which the parties and the adversary have access to an ideal version of a pseudo-random function. The environment has indirect access to this oracle through the adversary. In some cases, the parties will have access to more than one oracle. For example, when using a sub-protocol that has previously been proven secure in the universal composability framework, we can use its ideal functionality as an oracle (an ideal functionality is in fact no different from any other oracle). Notice that there is no difference between proving security of a real world protocol and a hybrid world protocol: The simulator in the ideal world still cannot use these oracles, it only uses the ideal functionality that we are trying to realize. In other words, the simulator must also simulate the oracles, so that the adversary's interface stays the same. Fig. 4 shows an overview of the hybrid world.

The random oracle model. The random oracle model [15] is a hybrid world model in which the parties and the adversary can query an idealized version of a pseudo-random function $\mathcal{R}$. The oracle returns a uniformly random value

when queried on an input it has never seen before. On a previously-queried input, it provides the same output. This model is a useful heuristic, as it allows for extracting secrets during simulation, which enables it to prove security for protocols that could not be proven secure in the standard model.

Instead of describing the random oracle as an oracle or ideal functionality, we provide the parties with a function $\mathtt{H}$. In particular, we provide the parties with multiple functions such as $\mathtt{H}_{\mathsf{K}}$ to model pseudo-random functions with different output distributions. Internally, these functions prepend a unique string before querying the random oracle. For domain separation, these functions also implicitly prepend a protocol string. For example, $\mathtt{H}_{\mathsf{K}}(\dots)$ in Protocol Π will query the random oracle as follows: $\mathtt{H}(\Pi, \mathsf{K}, \dots)$.

A special case is $\mathtt{H}_{\mathsf{pb}}$, which is a password-based key derivation function. Such a function models a PRF that is very expensive to compute. We model this as follows: The function first queries the random oracle as expected, i.e. $y \leftarrow \mathtt{H}(\Pi, \mathsf{pb}, \dots)$, and then performs a hard-to-invert and slow-to-compute univariate function on y.

4.2 Existing definitions

On a high level, all previously proposed ideal PAKE functionalities work as follows. Parties initiate a session by sending some message to the ideal functionality with the (potentially wrong) password and a session identifier. The session will only be finished when the adversary decides it to be, which models an adversary who controls the communication channels. The adversary can finish the session for a party by sending some message with the party's identifier and the session identifier to the ideal functionality, and the ideal functionality will send two potentially matching random session keys to the two parties.

At this point, the above functionality is powerful but practically unrealizable: since the password is only a low-entropy secret, we must model the fact that with non-negligible probability, the adversary might simply guess it. For this reason, the ideal functionality is equipped with another query that an adversary can make once per session for each of the two parties. This query requires the adversary to send the party identifier, the session identifier, and a password guess, modeling an online password guessing attack. If the password is guessed correctly, the session is considered compromised. If it was incorrect, the session is considered interrupted. Different variants handle an interrupted session differently, e.g. they abort or they also allow a party to receive a random key, but they handle compromised sessions the same, namely the adversary gets to select the key that is sent to the party.

The paradigm of Canetti et al. The functionality of Canetti et al. [26] and various follow-up works, e.g. the lazy extraction functionality of [1], let parties initiate a new session by sending $(\mathtt{NewSession}, \mathsf{sid}, \mathcal{P}, \mathcal{P}', \mathsf{pw})$ to the ideal functionality. It lets the adversary perform a guessing attack using $(\mathtt{TestPwd}, \mathsf{sid}, \mathcal{P}, \mathsf{pw}^*)$, and finish the session using $(\mathtt{NewKey}, \mathsf{sid}, \mathcal{P}, \mathsf{K}^*)$. If the session is compromised, the ideal functionality sends K^* to $\mathcal{P}$. Otherwise the session keys are randomly generated. The established session keys will match if the passwords match.

Shoup's paradigm. In the paradigm discussed above, a session will complete successfully (i.e. in a way that the established keys are random and matching) if the session was not compromised and the passwords match. However, this does not require two parties to always use the same password: the two parties can use a different password for every session as long as they match. Shoup's ideal functionality [74] is one for unbalanced PAKEs, in which parties first register with a password, and the server is guaranteed to always use that password. A second difference from the ideal functionality by Canetti et al. is that Shoup's ideal functionality implies key confirmation, i.e. entity authentication: If the protocol does not abort, the two parties have established matching keys.

Lazy extraction. Protocols like SPAKE2 or CPace are hard to simulate [1,74] because the simulator is required to extract the password during protocol execution (so before the parties locally compute the final hash) from a message that perfectly hides the password. Abdalla et al. [1] show that you can prove protocols like SPAKE2 or CPace secure in the universal composability model by relaxing Canetti et al.'s ideal functionality to allow for late password test. Such a password test is late in the sense that it comes after the session keys have been established or the protocol has aborted. One caveat is that the established key from such a protocol may never be deemed secure because the adversary may still succeed in attacking the protocol at any later time. However, the authors show that this can be overcome by adding key confirmation flows, after which the functionality implies perfect forward secrecy. In this model, the adversary must choose to either perform a password test during the protocol or after it finishes by registering a late password test before the protocol is over.

A problem with some functionalities. Abdalla et al. (see [4], Appendix C) point out that many derivations of the first ideal functionality given by Canetti et al. [26] include a flaw that makes them impractical for realizing secure communications. The ideal functionality correctly lets the adversary decide the established session key for an honest party if the adversary performs a successful online password attack. However, the functionality also allows the adversary to decide the the established session key for an honest party if either of the involved parties is dishonest.

Abdalla et al. discuss this problem in detail. Fortunately, it seems that no known protocols rely on this property for constructing the simulators in their proofs. The authors provide a fixed version of $\mathcal{F}_{\mathsf{lePAKE}}$ and show that CPace realizes it. The fix is simple: an adversary should only be able to decide the established session key for an honest party if it guesses the password correctly.

A note on forward secrecy. A commonly-desired property of key exchanges is forward secrecy, in which previously established keys remain secure even when a long-term key is leaked. In the context of PAKEs, this long-term key is the password. This property has been formalized by the BPR security game, and Abdalla et al. [1] show that $\mathcal{F}_{\mathsf{lePAKE}}$ followed by explicit key confirmation realizes $\mathcal{F}_{\mathsf{rPAKE\text{-}EA}}$, which is secure in the BPR model. As such, $\mathcal{F}_{\mathsf{lePAKE}}$ followed by explicit key confirmation provides perfect forward secrecy, while $\mathcal{F}_{\mathsf{lePAKE}}$ with-

out key confirmation provides weak forward secrecy. This latter notion implies forward secrecy only for sessions in which the adversary did not actively interfere. We argue that the analysis by Abdalla et al. extends to $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$: when the adversary performs a wrong late password test, it does not learn anything about the established session key.

At the same time, the BPR model has some clear drawbacks when compared to UC proofs [74]:

- The BPR model assumes that all passwords are independent samples from a specific distribution.
- The BPR model does not provide security guarantees after a user's password is broken.
- It is not straightforward to determine the security properties of a secure channel instantiated from a PAKE that is secure in the BPR model.

These drawbacks also mean that some protocols that may be proven secure in the BPR model might not be UC-secure. To the best of our knowledge, there is also no such security definition for PFS for asymmetric PAKEs.

4.3 Symmetric PAKEs with lazy extraction

We choose to use the ideal functionality that Shoup used to prove security for SPAKE2, as it has a simple transformation to aPAKEs, while incorporating relaxed lazy extraction as described by Hesse & Rosenberg [47] and transforming it to the balanced setting. Next to that, we remove the relationship identifier rid and instead incorporate a shared session identifier ssid. We present the resulting functionalities $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{LE}}}$ and $FSrle$ incrementally by starting from Shoup's original functionality. The detailed interfaces can be found in Appendix A.

$\mathcal{F}^*_{\mathsf{PAKE}^{(\mathsf{R})\mathsf{LE}}}$ and $\mathcal{F}_{\mathsf{PAKE}^{(\mathsf{R})\mathsf{LE}}}$

The ideal functionality is comprised of the initialization interface A1, the adversary's interface A4 and the lazy extraction interface A5.
$\mathcal{F}^*_{\mathsf{PAKE}^{[\mathsf{x}]}}$ and $\mathcal{F}_{\mathsf{PAKE}^{[\mathsf{x}]}}$ for $\mathsf{x} \in \{\mathsf{LE}, \mathsf{RLE}\}$ differ in the `passwords-matching` query.
Functionalities for $\mathsf{PAKE}^{\mathsf{LE}}$ and $\mathsf{PAKE}^{\mathsf{RLE}}$ differ in the `late-test-pwd` query.
We refer to Appendix A for the full version of all interfaces.

Balanced PAKE: initialization interface A1:

- $(\texttt{init-user-inst}, \mathsf{ssid}, S, \mathsf{pw})$ from user U: Instance U_{ssid} has state *original*
- $(\texttt{init-server-inst}, \mathsf{ssid}, U, \mathsf{pw})$ from server S: Instance S_{ssid} has state *original*

Balanced PAKE: adversary's interface A4:

- $(\texttt{passwords-matching}, \mathsf{ssid}, U, S)$: Instances U_{ssid} and S_{ssid} had state *original*, and now have state *eq-tested*(b) where $b = 1$ iff the two instances use matching passwords. The adversary learns bit b ($\mathcal{F}^*_{\mathsf{PAKE}^{[\mathsf{x}]}}$) or the bit is hidden ($\mathcal{F}_{\mathsf{PAKE}^{[\mathsf{x}]}}$)

- (`fresh-key`, ssid, X, ftr): Both X_{ssid} and its counterparty Y_{ssid} had state *eq-tested*(b), and now X_{ssid} outputs a random key denoted `fresh-key`
- (`copy-key`, ssid, Y): Instance Y_{ssid} had state *eq-tested*(b) and its counterparty X_{ssid} output a key marked `fresh-key`. If $b = 1$ then Y_{ssid} now outputs the same key, otherwise it outputs a random key
- (`test-pwd`, ssid, P, pw′): Instance P_{ssid} had state *original* and now has state *correct-guess* or *incorrect-guess*, depending on whether $\mathsf{pw}' = \mathsf{pw}$
- (`corrupt-key`, ssid, P, K, ftr): Instance P_{ssid} had state *correct-guess*, and now it outputs adversarial key K
- (`abort`, ssid, P): If P_{ssid} hasn't terminated then it aborts
- (`random-key`, ssid, P, ftr): If P_{ssid} hasn't terminated then it outputs a random key

(Relaxed) lazy extraction PAKE: adversary's interface A5:

- (`register-test`, ssid, P): P_{ssid} had state *original*, and now has state *incorrect-guess* flagged tested
- (`late-test-pwd`, ssid, P, pw′): P_{ssid} was flagged tested, and now the flag is removed and the adversary learns P's key if the guess is correct, otherwise it receives a random key ($\mathsf{PAKE}^{\mathsf{LE}}$) or a rejection symbol ($\mathsf{PAKE}^{\mathsf{RLE}}$).

Switching to the balanced setting. We start by presenting the initialization interface as it constitutes the main difference between a balanced and unbalanced PAKE. We completely remove user and server initialization to reflect that both parties can input a password, which differs from the unbalanced ideal functionality presented in [74], in which only the user provides a password. We track instances of sessions using shared session identifiers: No two instances of a given party share the same instance ID ssid.

Removing implicit key confirmation. Shoup's functionality only allows the protocol to output valid keys: if the passwords do not match, then the simulator can only make the protocol abort (or stall indefinitely). In $\mathcal{F}^{*}_{\mathsf{PAKE}^{\mathsf{LE}}}$ we also allow the simulator to make the protocol output random keys instead of aborting. The change between our functionality and the original functionality by Shoup is that we add a `random-key` query.

The reason that we do not tackle key confirmation at this stage is that we are looking for an ideal functionality for balanced PAKEs that we can compose into an augmented PAKE that achieves its key confirmation in a different way (i.e. we do not need to get it from the PAKE directly).

Adding lazy extraction. The original formulation of an ideal PAKE functionality $\mathcal{F}_{\mathsf{pwKE}}$ by Canetti et al. [26] only allows the adversary to make password guesses during the execution of the protocol. However, as explained by Abdalla et al. [31], one cannot simulate a Diffie-Hellman based PAKE that ends with a hash function in this way since the simulator cannot extract the password guess during the protocol execution. To address this problem, Abdalla [1] proposes a different functionality $\mathcal{F}_{\mathsf{lePAKE}}$ that provides the attacker with additional capabilities, allowing it to register a late password guess and perform one at a later time. However, to make sure that the key can ever be considered uncompromised, one needs to perform key confirmation. Shoup [74] also noticed this problem and

instead incorporated key confirmation directly into the ideal functionality. We present both $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{LE}}}$ and $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$, which differ in how they reply to incorrect late password tests, similar to [47]. On a correct guess, both functionalities leak the session key to the adversary. However, on a wrong guess, $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{LE}}}$ replies with a random session key, whereas $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$ replies with a rejection symbol $\perp$.

4.4 Asymmetric PAKEs with explicit authentication

Our vision now shifts to extending post-quantum and hybrid PAKEs to augmented/asymmetric PAKEs. Such a PAKE protects the user's password against server compromise. We must first define an ideal functionality for such a protocol. In this section, we present a slightly altered ideal functionality of the one by Shoup [74]. The functionality is unbalanced in the sense that the server is assumed to always use the correct password. It also implicitly enforces key confirmation: it is not possible to end up with different established session keys in an uncompromised session. We use a different variant of lazy extraction here called 'relaxed', as proposed by Abdalla et al. [1]. On a late password test, the functionality only replies whether the guess was correct or not. Note that this differs from relaxed lazy extraction. We present detailed functionalities in Appendix A.

Modeling password-based key derivation functions. We follow a similar technique here as used by Shoup [74] to transform his ideal PAKE functionality to one for aPAKEs. The main problem in designing such a functionality is in modeling the verifier stored at the server (and which can be obtained by an adversary if it compromises the server). The verifier is nothing more than a hash that is somewhat inefficient to compute. The problem is that an attacker can recover the password from a verifier in polynomial time by running through a polynomial-size dictionary, so we have to limit the way in which the attacker can query the random oracle. Shoup does this by letting the environment choose when and on what inputs the adversary can query this oracle. In the real world, the environment sends a message $(\texttt{oracle-query}, \mathsf{rid}, \mathsf{pw}')$ to the random oracle who sends $(\texttt{oracle-query}, \mathsf{rid}, \mathsf{pw}', \mathtt{H}(\mathsf{rid}, \mathsf{pw}'))$ to the adversary. In the ideal world, any simulator must also simulate the random oracle. For this reason, the ideal functionality accepts a message $(\texttt{oracle-query}, \mathsf{rid}, \mathsf{pw}')$ from the environment and forwards it to the simulator, who then gets to call $(\texttt{offline-test-pwd}, S, \mathsf{pw}')$ and send a simulated result to the adversary. The aPAKE functionality also includes a function (`compromise-pwdfile`), which models that the adversary obtains the password file.

Making the PAKE unbalanced. Next, we consider that an aPAKE is unbalanced, in the sense that the server always uses the correct password. The user may still use any passwords, to model that passwords can be forgotten or mistyped. We provide such an unbalanced initialization interface below.

The ideal functionality. We now present the composed ideal functionality. We make two additional changes here:

1. The ideal functionality no longer has a `random-key` query, which models key confirmation. Uncompromised sessions must therefore abort if the established session keys do not match.
2. We use the relaxed PAKE lazy extraction technique from Abdalla et al. [1]. The output of `late-test-pwd` queries is no longer the actual session key, but a single message whether the guessed password was `correct` or `incorrect`. Abdalla et al. show for symmetric PAKEs that this notion still achieves perfect forward secrecy. This corresponds to the 'leaky' functionality proposed by Lyu & Liu [60].

We refer to this new ideal functionality as $\mathcal{F}^{*}_{\mathsf{aPAKE\text{-}EA^R}}$. We provide a summary below, while the full functionality is displayed in Appendix A.

$\mathcal{F}^{*}_{\mathsf{aPAKE\text{-}EA^R}}$

The ideal functionality is comprised of the initialization interface A2, the adversary's interface A4, the lazy extraction interface A5, and the server compromise interface A3. We briefly summarize their queries. See Appendix A for the full ideal functionalities.

Unbalanced PAKE: initialization interface A2:

- (`init-user`, rid, S, uid, pw) from user U: U starts account establishment
- (`init-server`, rid, U, uid) from server S: S starts account establishment
- (`init-fin`, rid) from the adversary: S creates password file `pwdfile`
- (`init-user-inst`, ssid, S, rid, uid, pw) from user U: Instance U_{ssid} has state *original*
- (`init-server-inst`, ssid, U, rid, uid) from server S: Instance S_{ssid} has state *original*

Server compromise interface A3:

- (`compromise-pwdfile`, S, rid) from the environment: `pwdfile` is now *compromised*
- (`oracle-query`, S, rid, uid$'$, pw$'$) from the environment: Forwards the query to $\mathcal{A}$
- (`offline-test-pwd`, S, rid, uid$'$, pw$'$) from the adversary: If `pwdfile` is *compromised* then the adversary learns if pw$'$ = pw

Unbalanced PAKE with key confirmation: adversary's interface A4:

- (`passwords-matching`, ssid, U, S): Instances U_{ssid} and S_{ssid} had state *original*, and now have state *eq-tested*(b) and the adversary learns bit b where $b = 1$ iff the two instances use matching passwords
- (`fresh-key`, ssid, X, ftr): Both X_{ssid} and its counterparty Y_{ssid} had state *eq-tested*(b), and now X_{ssid} outputs a random key denoted `fresh-key`
- (`copy-key`, ssid, Y): Instance Y_{ssid} had state *eq-tested*(b) and its counterparty X_{ssid} output a key marked `fresh-key`. If $b = 1$ then Y_{ssid} now outputs the same key, otherwise it outputs a random key
- (`test-pwd`, ssid, P, pw$'$): Instance P_{ssid} had state *original* and now has state *correct-guess* or *incorrect-guess*, depending on whether pw$'$ = pw
- (`impersonate`, ssid, U, S, rid): If `pwdfile` was compromised and U_{ssid} had state *original*, now it has state *correct-guess*/*incorrect-guess* depending on whether the password in `pwdfile` matches the one used by U_{ssid}
- (`corrupt-key`, ssid, P, K, ftr): Instance P_{ssid} had state *correct-guess*, and now it outputs adversarial key K
- (`abort`, ssid, P): If P_{ssid} hasn't terminated then it aborts.

Relaxed PAKE: adversary's interface A5:

- $(\texttt{register-test}, \mathsf{ssid}, P)$: P_{ssid} had state *original*, and now has state *incorrect-guess* flagged tested
- $(\texttt{late-test-pwd}, \mathsf{ssid}, P, \mathsf{pw}')$: P_{ssid} was flagged tested, and now the flag is removed and the adversary learns if the guess is correct

5 Security analysis

5.1 Security of OQUAKE

For our security analysis of OQUAKE, we rely entirely on the proof by Arriaga et al. [9]. The authors proved that NoIC achieves $\mathcal{F}_{\mathsf{pwKE}}$ when the KEM satisfies the following three properties:

- It is One-Way (OW) under a Plaintext Checking Attack (PCA)
- Its public keys are uniform: i.e. they appear like uniformly random elements
- Its ciphertexts are anonymous (ANO) under the PCA attack, i.e. the ciphertexts cannot be related to the public key that created them.

All properties are achieved by ML-KEM under the M-LWE assumption. OQUAKE can be seen as a specific case of NoIC where the public key space is the set of λ_{pk}-bit bitstrings. One can achieve this by mapping the ML-KEM public keys using a bijective encoding such as Kemeleon [43]. As such, OQUAKE achieves $\mathcal{F}_{\mathsf{pwKE}}$, and in Appendix B we show that this in particular implies $\mathcal{F}_{\mathsf{PAKE}}$.

5.2 Security of CPaceOQUAKE

The sequential composition of two PAKEs has been studied in recent works by Hesse & Rosenberg [47] and Lyu & Liu [60]. Hesse & Rosenberg's sequential combiner composes a statistically password-hiding PAKE with one that statistically hides whether the two passwords matched. Unfortunately, OQUAKE does not necessarily statistically hide whether the two passwords matched; when instantiated with ML-KEM, an attacker that can break D-MLWE can check whether a ciphertext is valid or not. If the passwords do not match, the ciphertext is almost never valid. The composition proposed by Lyu & Liu is very similar, but instead of generalizing the first PAKE to all password-hiding PAKEs, it focuses on Diffie Hellman-type PAKEs specifically. These DH-type PAKEs require two rounds and are required to satisfy a similar password hiding property. The authors also show that sequential composition of such a DH-type PAKE with a PAKE secure in the QROM realizes a hybrid PAKE that is secure in the QROM. However, the proofs only apply when parties input the correct, matching passwords. To fill the gaps in existing proofs, we provide security proofs that also apply when the passwords do not match, and without assuming that OQUAKE hides password equality. We rely on the fact that CPace has previously been

shown to realize $\mathcal{F}_{\mathsf{lePAKE}}$ [4], and by extension $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ (see Appendix B), and that OQUAKE has been shown to realize $\mathcal{F}_{\mathsf{pwKE}}$ [9], and by extension $\mathcal{F}_{\mathsf{PAKE}}$.

Our proof strategy here is to show two things:

1. Take a protocol that realizes a lazy-extraction PAKE functionality $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ to establish keys K and K', and then run an arbitrary (even insecure) PAKE on inputs derived from these keys. We prove that this protocol realizes a relaxed lazy-extraction PAKE functionality $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$.
2. Run protocol CPace to establish keys K and K', and then run any secure PAKE on inputs derived from these keys. We prove that this protocol still realizes a secure PAKE even in the presence of a discrete logarithm oracle (which would make CPace insecure by itself).

If these two statements hold, then our protocol realizes a hybrid PAKE.

We first prove that protocol CPaceOQUAKE, i.e. protocol 3, withstands classical adversaries that can break the KEM, and by extension, protocol OQUAKE.

In the advantage bound below, the terms $\frac{q_{\mathsf{H}}}{|\mathbb{K}|}$ refer to probability that the key generated by CPace has been used as an input for one of the hash functions before. The terms following it originate from the probability that, in a session, the key output by the KEM is incorrect, that the adversary may guess the authenticator h_1, or that it simply guesses the PAKE key.

Theorem 1. *Protocol CPaceOQUAKE, i.e. protocol 3, realizes functionality $\mathcal{F}^*_{\mathit{PAKE}^{\mathit{RLE}}}$ in the random oracle model if CPace realizes functionality $\mathcal{F}_{\mathit{PAKE}^{\mathit{LE}}}$ and if* KDF *is a secure PRG.*

If $\mathrm{Adv}^{\mathrm{CPace}}_{\mathsf{lePAKE}}(\mathcal{A}_1)$ *is an advantage of adversary $\mathcal{A}_1$ against the $\mathcal{F}_{\mathit{lePAKE}}$ notion for protocol CPace, and* $\mathrm{Adv}^{\mathrm{KDF}}_{\mathsf{PRG}}(\mathcal{A}_2)$ *is an advantage of adversary $\mathcal{A}_2$ against the PRG security of* KDF*, then the advantage of any adversary $\mathcal{A}'$ with comparable resources to $\mathcal{A}_1$ and $\mathcal{A}_2$ taken together, against $\mathcal{F}^*_{\mathit{PAKE}^{\mathit{RLE}}}$ notion of protocol 3, is bounded by:*

$$\mathrm{Adv}^{prot\text{-}3}_{\mathsf{rlePAKE}^*}(\mathcal{A}') \leq \mathrm{Adv}^{\mathrm{CPace}}_{\mathsf{lePAKE}}(\mathcal{A}_1) + \mathrm{Adv}^{\mathrm{KDF}}_{\mathsf{PRG}}(\mathcal{A}_2) + \frac{3}{2^{\kappa}}$$

Sketch. We provide a proof in Appendix C. The proof follows by an argument which introduces a slight modification in the proof of a lemma given by Hesse and Rosenberg [47], which argues that this sequential compiler realizes functionality $\mathcal{F}_{\mathsf{rlePAKE}}$ in the case where PAKE #2 satisfies the (statistical) *pre-shared key equality-hiding* (PSK-EH) property and PAKE #1 realizes functionality $\mathcal{F}_{\mathsf{lePAKE}}$. We first note that by Theorem 4 in Appendix B, any protocol that realizes $\mathcal{F}_{\mathsf{rlePAKE}}$ also realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$. Second, the only change we must make in the argument of [47] is that in order to emulate PAKE #2 which does not have the PSK-EH property, i.e. if PAKE #2 is OQUAKE, the simulator needs to know if two parties who are passively connected in PAKE #1, i.e. in CPace, used matching passwords, in which case they output the same random key K_1, or they used different passwords, in which case U and S output independent random values as K_1. This is precisely the difference between functionality $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$,

realized if PAKE #2 is PSK-EH, and functionality $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$, realized by arbitrary PAKE #2: The latter functionality lets the simulator learn whether two passively-connected sessions have equal inputs (and therefore if they will or will not create a shared secure session), and therefore allows the simulator to emulate an arbitrary PAKE #2 protocol on the inputs derived from PAKE #1 outputs.

Next, we argue that if PAKE #2, i.e. OQUAKE, is secure against quantum adversaries, then CPaceOQUAKE, i.e. protocol 3, withstands quantum adversaries that can break the discrete logarithm problem and thus render render PAKE #1, i.e. CPace, insecure on its own. This follows by the analysis of Hesse & Rosenberg [47] which shows that the a stronger statement holds, i.e. that if OQUAKE realizes $\mathcal{F}_{\mathsf{PAKE}}$ then so does CPaceOQUAKE, but we show the theorem below as an independent validation of the same protocol.

Theorem 2. *Protocol CPaceOQUAKE, i.e. protocol 3, realizes functionality $\mathcal{F}_{PAKE}$ in the random oracle model if OQUAKE realizes functionality $\mathcal{F}_{pwKE}$.*

If $\mathrm{Adv}^{\mathrm{OQUAKE}}_{\mathsf{pwKE}}(\mathcal{A})$ is an advantage of adversary $\mathcal{A}$ against the $\mathcal{F}_{pwKE}$ notion of OQUAKE then the advantage of any adversary $\mathcal{A}'$ with comparable resources to $\mathcal{A}$, against the $\mathcal{F}_{PAKE}$ notion of protocol 3, is bounded by:

$$\mathrm{Adv}^{prot\text{-}3}_{\mathsf{PAKE}}(\mathcal{A}') \leq \mathrm{Adv}^{\mathrm{OQUAKE}}_{\mathsf{pwKE}}(\mathcal{A}) + \frac{2}{2^\kappa}$$

Sketch. We provide a proof in Appendix C, but this proof is a simple corollary of Theorem 2 in [47], which implies if OQUAKE realizes $\mathcal{F}_{\mathsf{pwKE}}$ then so does protocol 3, and Theorem 4 in Appendix B, which says that if a protocol realizes $\mathcal{F}_{\mathsf{pwKE}}$ then it also realizes $\mathcal{F}_{\mathsf{PAKE}}$.

5.3 Security of CPaceOQUAKE+

As discussed in Sections 1 and 2.3, there have been two previous analyses of similar PAKE(+KEM)-to-aPAKE compilers, by Gu [40] and Lyu et al. [61], and our PAKE-to-aPAKE compiler, CPaceOQUAKE+ shown as Protocol 4, can be thought of as a variant of these. However, there are three main differences from the compilers of [40,61]: (1) Whereas these compilers use a generic authenticated encryption with additional properties, e.g. random key robustness, for delivery of KEM ciphertext c on the PAKE-established secure channel represented by key K, we use a specific instantiation of authenticated encryption, namely one-time pad encryption with a MAC computed via a random oracle hash. This implements an authenticated encryption with random key robustness, although we do not explicitly model it as such, but it also has a stronger bounding to the PAKE transcript, and consequently implements a server-to-user entity authentication. (2) Secondly, whereas [40,61] assume that the symmetric PAKE realizes the standard PAKE functionality $\mathcal{F}_{\mathsf{pwKE}}$, and hence also $\mathcal{F}_{\mathsf{PAKE}}$ (by Theorem 4 in Appendix B), we must start from a lazy-extraction PAKE that leaks password equality, specifically from any realization of functionality $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$. As such, the resulting aPAKE achieves a slightly weaker notion of security defined by an

ideal functionality $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$, see Section 4.4. (3) Finally, the asymmetric PAKE functionality $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ differs from the UC aPAKE notion used in [40,61] in that it supports interactive account initialization, and it implies secrecy of not only a password but also of the user account identifier.

In the theorem below we claim protocol 4 realizes functionality $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$, if the underlying symmetric PAKE realizes $\mathcal{F}^*_{\mathsf{PAKE^{RLE}}}$, KEM is CCA-secure, and hash functions $\mathtt{H_{pb}}, \mathtt{H}_c, \mathtt{H}_1, \mathtt{H}_2$ are modeled as Random Oracles, but we only prove it under the assumptions that the account initialization phase was done securely. Specifically, we make two assumptions: First, we assume that party U and S engaged in account initialization, i.e. the parties that receive resp. $(\mathtt{init\text{-}user}, \mathsf{rid}, S, \mathsf{uid}, \mathsf{pw})$ and $(\mathtt{init\text{-}server}, \mathsf{rid}, U, \mathsf{uid})$ queries from the environment, are *not corrupted during initialization*, and in particular that they honestly follow the prescribed protocol reacting to these queries. Second, we assume that the initialization protocol between U and S proceeds over *a secure channel*. Note that the initialization protocol consists of U sending a single message to S, namely a tuple $(\mathsf{rid}, S, \mathsf{uid}, v, pk)$, which S accepts only if fields $\mathsf{rid}, S, \mathsf{uid}$ match its inputs $(\mathsf{rid}, \mathsf{uid})$ and its own identity S. The assumption we make is that this tuple is transferred over a secure, i.e. confidential and authenticated, channel between U and S, so the adversary doesn't learn anything about this tuple and cannot modify it. The adversary can only either let this message pass or stop it, which is reflected in the $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ model by the adversary respectively sending query $(\mathtt{init\text{-}fin}, \mathsf{rid})$ to $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ or not sending it.

Theorem 3. *The PAKE-to-aPAKE compiler protocol CPaceOQUAKE+, i.e. protocol 4, realizes functionality $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ in the random oracle model if the inner symmetric PAKE realizes functionality $\mathcal{F}^*_{\mathsf{PAKE^{RLE}}}$ and KEM is CCA-secure, and assuming that the account initialization was done by honest parties over a secure channel.*

If $\mathrm{Adv}^{\mathrm{inner}}_{\mathsf{PAKE^{RLE}}}(\mathcal{A}_1)$ *is an advantage of adversary* $\mathcal{A}_1$ *against the* $\mathcal{F}^*_{\mathsf{PAKE^{RLE}}}$ *notion of the inner symmetric PAKE and* $\mathrm{Adv}^{\mathsf{KEM}}_{\mathsf{ind\text{-}cca}}(\mathcal{A}_2)$ *is an advantage of adversary* $\mathcal{A}_2$ *against the IND-CCA property of* KEM*, then the advantage of any protocol 4 adversary* $\mathcal{A}'$ *with comparable resources to* $\mathcal{A}_1$ *and* $\mathcal{A}_2$ *taken together, against the* $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ *notion of protocol 4, is bounded by:*

$$\mathrm{Adv}^{prot\text{-}4}_{\mathsf{aPAKE\text{-}EA^R}}(\mathcal{A}') \leq \mathrm{Adv}^{\mathrm{inner}}_{\mathsf{PAKE^{RLE}}}(\mathcal{A}_1) + q^2_{\mathsf{pb}} \cdot 2^{-\lambda_v - 1} + q_s \cdot (2^{-\lambda_v + 2} + 2^{-\lambda_h + 2} + \mathrm{Adv}^{\mathsf{KEM}}_{\mathsf{ind\text{-}cca}}(\mathcal{A}_2)) \; .$$

Sketch. We provide a proof in Appendix D, and here we briefly explain the intuition.

In the first part of the proof, we must show that we can simulate the behavior of the PAKE ideal functionality ($\mathcal{F}^*_{\mathsf{PAKE^{LE}}}$) using only the aPAKE ideal functionality ($\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$). For the most part, the simulator can perform the operations that the PAKE ideal functionality does, such as generating session keys. However, notice that there are two crucial differences between the ways these interfaces behave in the protocol. Firstly, in the PAKE, the parties use verifiers for their password, whereas they use their own password in the aPAKE.

For the simulator to answer `test-pwd` queries correctly, it must therefore find preimages for the verifiers. It does so by simulating $\mathtt{H}_{\mathsf{pb}}$ (which is used to create the verifiers) in such a way that the outputs are unique for each password. The simulator can then find the corresponding password by checking the state of $\mathtt{H}_{\mathsf{pb}}$'s simulation. Secondly, the aPAKE provides a different form of lazy extraction. When the aPAKE has not yet finished, this is not a problem, because late password queries to $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{LE}}}$ can be simulated using regular `test-pwd` queries to $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA}^{\mathsf{R}}}$. Otherwise, the simulator must make a `late-test-pwd` call to $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA}^{\mathsf{R}}}$, which returns `correct` or `incorrect`. Based on this result, the simulator can reply to the adversary with a simulated K or a randomly-generated one.

One problem with the simulation of $\mathtt{H}_{\mathsf{pb}}$ above, is that it must remain consistent when the server is compromised. We use the same strategy as Shoup [74]. The idea is that upon server compromise, we check whether any of the password queries that have been made so far were for the correct password. The simulator uses `offline-test-pwd` for this, which it can now use as S has been compromised. If a correct password query was made, the simulator knows $v||d$, so it returns v and pk to $\mathcal{A}$, where $(pk, sk) \leftarrow \mathsf{KEM}.\mathrm{KeyGen}(d)$. Otherwise, it samples random v and d, but it must change its future behavior: If at a future time a correct password query is made, the simulator must ensure that it replies with the v and d that it generated upon compromise.

At this point, the main challenge to solve is for the simulator to decide when it should abort, and when it should establish keys. Recall that the aPAKE ideal functionality does not allow random non-matching keys to be established in uncompromised sessions. If the session keys established by the PAKE are compromised, then the simulator can extract all the private values that it needs to proceed with the protocol as usual. In other cases, if a message comes from the adversary, the simulator aborts. We show that as long as the KEM satisfies IND-CCA2, an adversary cannot come up with authentication tags that pass authentication.

6 Conclusion

In this work, we proposed PAKEs that resist attacks from quantum attackers. One of the main benefits of these protocols is that they can be instantiated using standardized primitives. We provide security proofs for all our protocols in the universal composability framework in the random oracle model. A crucial next step for theoretical security is to prove security in the quantum random oracle model. We believe that this is possible using compressed quantum random oracles, because the simulated random oracles only rely on historical queries in simple ways. Besides this, there are many directions for future work. One might:

- Consider alternative constructions, e.g. using SPAKE2 or OQUAKE based on a classical UKEM.
- Extend our security proofs to adaptive corruption.
- Provide a proof of our protocols in the BPR model.

- Extend our security proofs to the bare PAKE model.
- Extend the design to avoid the RPE-PCS relaxation in the PAKE model. For example, by password-encrypting the second flow of OQUAKE.

Finally, it would be useful to reduce the number of interactions of the hybrid (a)PAKE to only three rounds while providing mutual authentication and without significantly increasing computation and communication. The first proposal for such a PAKE is by Günther et al. [42], which uses an adaptation of FrodoKEM with fully random public keys to achieve a password-hiding post-quantum instantiation of an EKE-style PAKE. This PAKE can be composed in parallel with a classical password-hiding PAKE to achieve a hybrid solution. Unfortunately, the adapted version of FrodoKEM requires ciphertexts of 144 kilobytes.

References

1. Michel Abdalla, Manuel Barbosa, Tatiana Bradley, Stanislaw Jarecki, Jonathan Katz, and Jiayu Xu. Universally composable relaxed password authenticated key exchange. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 278–307. Springer, 2020. 1, 1, 4, 6, 1, 4.2, 4.3, 4.4, 2, A, B, B, 1, **??**, 5, B
2. Michel Abdalla, Fabrice Benhamouda, and David Pointcheval. Public-key encryption indistinguishable under plaintext-checkable attacks. In Jonathan Katz, editor, *Public-Key Cryptography - PKC 2015 - 18th IACR International Conference on Practice and Theory in Public-Key Cryptography, Gaithersburg, MD, USA, March 30 - April 1, 2015, Proceedings*, volume 9020 of *Lecture Notes in Computer Science*, pages 332–352. Springer, 2015. 1, 2.2, 2.2, 2.2
3. Michel Abdalla, Thorsten Eisenhofer, Eike Kiltz, Sabrina Kunzweiler, and Doreen Riepel. Password-authenticated key exchange from group actions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 699–728. Springer, 2022. 1
4. Michel Abdalla, Björn Haase, and Julia Hesse. Security analysis of cpace. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 711–741. Springer, 2021. 1, 3.2, 6, 4, 4.2, 5.2, A, 1
5. Michel Abdalla and David Pointcheval. Simple password-based encrypted key exchange protocols. In Alfred Menezes, editor, *Topics in Cryptology - CT-RSA 2005, The Cryptographers' Track at the RSA Conference 2005, San Francisco, CA, USA, February 14-18, 2005, Proceedings*, volume 3376 of *Lecture Notes in Computer Science*, pages 191–208. Springer, 2005. 1
6. Navid Alamati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security,*

Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II, volume 12492 of *Lecture Notes in Computer Science*, pages 411–439. Springer, 2020. 2.1
7. Erdem Alkim, Joppe W. Bos, Léo Ducas, Patrick Longa, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Chris Peikert, Ananth Raghunathan, and Douglas Stebila. *FrodoKEM: Learning With Errors Key Encapsulation*, 2021. 2.1
8. Nouri Alnahawi, Kathrin Hövelmanns, Andreas Hülsing, Silvia Ritsch, and Alexander Wiesmaier. Towards post-quantum secure PAKE - a tight security proof for ocake in the bpr model. Cryptology ePrint Archive, Paper 2023/1368, 2023. https://eprint.iacr.org/2023/1368. 1, 1, 2.2
9. Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. Cryptology ePrint Archive, Paper 2025/231, 2025. 1, 1, 1, 3, 3.1, 1, 5.1, 5.2, A
10. Afonso Arriaga, Manuel Barbosa, Stanislaw Jarecki, and Marjan Skrobot. C'est très CHIC: A compact password-authenticated key exchange from lattice-based KEM. *IACR Cryptol. ePrint Arch.*, page 308, 2024. 1, 1, 2.2
11. Boaz Barak, Ran Canetti, Yehuda Lindell, Rafael Pass, and Tal Rabin. Secure computation without authentication. In Victor Shoup, editor, *Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005, Proceedings*, volume 3621 of *Lecture Notes in Computer Science*, pages 361–377. Springer, 2005. 2.2
12. Manuel Barbosa, Kai Gellert, Julia Hesse, and Stanislaw Jarecki. Bare PAKE: Universally composable key exchange from just passwords. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 183–217. Springer, Cham, August 2024. 2.1, 1, 2.2, A
13. Hugo Beguinet, Céline Chevalier, David Pointcheval, Thomas Ricosset, and Mélissa Rossi. Get a CAKE: generic transformations from key encaspulation mechanisms to password authenticated key exchanges. In Mehdi Tibouchi and Xiaofeng Wang, editors, *Applied Cryptography and Network Security - 21st International Conference, ACNS 2023, Kyoto, Japan, June 19-22, 2023, Proceedings, Part II*, volume 13906 of *Lecture Notes in Computer Science*, pages 516–538. Springer, 2023. 1, 2.1, 1, 2.2, 2.2, 4
14. Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding*, volume 1807 of *Lecture Notes in Computer Science*, pages 139–155. Springer, 2000. 2.2
15. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security, Fairfax, Virginia, USA, November 3-5, 1993*, pages 62–73. ACM, 1993. 4.1
16. Steven M. Bellovin and Michael Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *1992 IEEE Symposium on Security and Privacy*, pages 72–84. IEEE Computer Society Press, May 1992. 1
17. Steven M. Bellovin and Michael Merritt. Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security, Fairfax, Virginia, USA, November 3-5, 1993*, pages 244–250. ACM, 1993. 1, 2.2

18. Fabrice Benhamouda, Olivier Blazy, Léo Ducas, and Willy Quach. Hash proof systems over lattices revisited. In Michel Abdalla and Ricardo Dahab, editors, *Public-Key Cryptography - PKC 2018 - 21st IACR International Conference on Practice and Theory of Public-Key Cryptography, Rio de Janeiro, Brazil, March 25-29, 2018, Proceedings, Part II*, volume 10770 of *Lecture Notes in Computer Science*, pages 644–674. Springer, 2018. 2.1
19. Nina Bindel, Jacqueline Brendel, Marc Fischlin, Brian Goncalves, and Douglas Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pages 206–226, 2019. 1
20. Nina Bindel, Udyani Herath, Matthew Mckague, and Douglas Stebila. Transitioning to a quantum-resistant public key infrastructure. In *International Workshop on Post-Quantum Cryptography*, pages 384–405, 06 2017. 1
21. Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology - ASIACRYPT 2011 - 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4-8, 2011. Proceedings*, volume 7073 of *Lecture Notes in Computer Science*, pages 41–69. Springer, 2011. 1
22. Joppe Bos, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. CRYSTALS - Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy – EuroS&P 2018*, pages 353–367, Los Alamitos, CA, USA, 2018. IEEE Computer Society. 1, 1, 3.1
23. Tatiana Bradley, Jan Camenisch, Stanislaw Jarecki, Anja Lehmann, Gregory Neven, and Jiayu Xu. Password-authenticated public-key encryption. In Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, editors, *Applied Cryptography and Network Security - 17th International Conference, ACNS 2019, Bogota, Colombia, June 5-7, 2019, Proceedings*, volume 11464 of *Lecture Notes in Computer Science*, pages 442–462. Springer, 2019. 1, 2.2
24. Emmanuel Bresson, Olivier Chevassut, and David Pointcheval. Security proofs for an efficient password-based key exchange. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent Jaeger, editors, *Proceedings of the 10th ACM Conference on Computer and Communications Security, CCS 2003, Washington, DC, USA, October 27-30, 2003*, pages 241–250. ACM, 2003. 2.2
25. Ran Canetti, Dana Dachman-Soled, Vinod Vaikuntanathan, and Hoeteck Wee. Efficient password authenticated key exchange via oblivious transfer. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *Public Key Cryptography - PKC 2012 - 15th International Conference on Practice and Theory in Public Key Cryptography, Darmstadt, Germany, May 21-23, 2012. Proceedings*, volume 7293 of *Lecture Notes in Computer Science*, pages 449–466. Springer, 2012. 1, 2.2, 2.2
26. Ran Canetti, Shai Halevi, Jonathan Katz, Yehuda Lindell, and Philip D. MacKenzie. Universally composable password-based key exchange. In Ronald Cramer, editor, *Advances in Cryptology - EUROCRYPT 2005, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings*, volume 3494 of *Lecture Notes in Computer Science*, pages 404–421. Springer, 2005. 4, 6, 1, 4.2, 4.3, A, B, B, **??**, 5, B
27. Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao. On the compressed-oracle technique, and post-quantum security of proofs of sequential

work. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 598–629. Springer, 2021. 3.1

28. D. Connolly, P. Schwabe, and B. Westerbaan. X-Wing: general-purpose hybrid post-quantum kemmodule-lattice-based key-encapsulation mechanism standard. Technical Report Federal Information Processing Standards Publications (FIPS PUBS) 203, CFRG Draft, 2024. 3
29. Jintai Ding, Saed Alsayigh, Jean Lancrenon, Saraswathy RV, and Michael Snook. Provably secure password authenticated key exchange based on RLWE for the post-quantum world. In Helena Handschuh, editor, *Topics in Cryptology - CT-RSA 2017 - The Cryptographers' Track at the RSA Conference 2017, San Francisco, CA, USA, February 14-17, 2017, Proceedings*, volume 10159 of *Lecture Notes in Computer Science*, pages 183–204. Springer, 2017. 1, 2.2
30. Bruno Freitas Dos Santos, Yanqi Gu, Stanislaw Jarecki, and Hugo Krawczyk. Asymmetric PAKE with low computation and communication. In *Advances in Cryptology – EUROCRYPT 2022: 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 – June 3, 2022, Proceedings, Part II*, page 127–156, Berlin, Heidelberg, 2022. Springer-Verlag. 1
31. Edward Eaton and Douglas Stebila. The "quantum annoying" property of password-authenticated key exchange protocols. In Jung Hee Cheon and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20-22, 2021, Proceedings*, volume 12841 of *Lecture Notes in Computer Science*, pages 154–173. Springer, 2021. 1, 2, 4.3
32. Edward Eaton and Douglas Stebila. The "quantum annoying" property of password-authenticated key exchange protocols. In Jung Hee Cheon and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20-22, 2021, Proceedings*, volume 12841 of *Lecture Notes in Computer Science*, pages 154–173. Springer, 2021. 3.2
33. Phillip Gajland, Bor de Kock, Miguel Quaresma, Giulio Malavolta, and Peter Schwabe. SWOOSH: efficient lattice-based non-interactive key exchange. In Davide Balzarotti and Wenyuan Xu, editors, *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. USENIX Association, 2024. 2.1
34. Xinwei Gao, Jintai Ding, Lin Li, Saraswathy RV, and Jiqiang Liu. Efficient implementation of password-based authenticated key exchange from RLWE and post-quantum TLS. *Int. J. Netw. Secur.*, 20(5):923–930, 2018. 2.2
35. Rosario Gennaro and Yehuda Lindell. A framework for password-based authenticated key exchange. In Eli Biham, editor, *Advances in Cryptology - EUROCRYPT 2003, International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4-8, 2003, Proceedings*, volume 2656 of *Lecture Notes in Computer Science*, pages 524–543. Springer, 2003. 1, 2.2, 2.2
36. Craig Gentry, Philip D. MacKenzie, and Zulfikar Ramzan. A method for making password-based key exchange resilient to server compromise. In Cynthia Dwork, editor, *Advances in Cryptology - CRYPTO 2006, 26th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 2006, Proceed-*

ings, volume 4117 of *Lecture Notes in Computer Science*, pages 142–159. Springer, 2006. 1, 2, 2.3, 2.3
37. Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 190–218, March 2018. 1
38. Adam Groce and Jonathan Katz. A new framework for efficient password-based authenticated key exchange. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *Proceedings of the 17th ACM Conference on Computer and Communications Security, CCS 2010, Chicago, Illinois, USA, October 4-8, 2010*, pages 516–525. ACM, 2010. 1, 2.2, 2.2, 2.2
39. Paul Grubbs, Varun Maram, and Kenneth G. Paterson. Anonymous, robust post-quantum public key encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 402–432. Springer, Cham, May / June 2022. 1, 3.1
40. Yanqi Gu. *New Paradigms For Efficient Password Authentication Protocols*. PhD thesis, UC Irvine, 2024. 1, 3, 1, 2.3, 2, 2.3, 3, 3.3, 5.3, A, D
41. Yanqi Gu, Stanislaw Jarecki, and Hugo Krawczyk. *KHAPE: Asymmetric PAKE from Key-Hiding Key Exchange*, pages 701–730. Springer-Verlag, 2021. 1
42. Felix Günther, Michael Rosenberg, Douglas Stebila, and Shannon Veitch. Hybrid obfuscated key exchange and KEMs. Cryptology ePrint Archive, Paper 2025/408, 2025. 3.2, 6
43. Felix Günther, Douglas Stebila, and Shannon Veitch. Obfuscated key exchange. *IACR Cryptol. ePrint Arch.*, page 1086, 2024. 1, 2.1, 5, 5.1
44. Björn Haase and Benoît Labrique. Aucpace: Efficient verifier-based PAKE protocol tailored for the iiot. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(2):1–48, 2019. 3.2, 2, 5
45. Julia Hesse. Separating standard and asymmetric password-authenticated key exchange. *IACR Cryptol. ePrint Arch.*, page 1064, 2019. 2.3
46. Julia Hesse. Separating symmetric and asymmetric password-authenticated key exchange. In Clemente Galdi and Vladimir Kolesnikov, editors, *Security and Cryptography for Networks - 12th International Conference, SCN 2020, Amalfi, Italy, September 14-16, 2020, Proceedings*, volume 12238 of *Lecture Notes in Computer Science*, pages 579–599. Springer, 2020. 6
47. Julia Hesse and Michael Rosenberg. PAKE combiners and efficient post-quantum instantiations. *IACR Cryptol. ePrint Arch.*, page 1621, 2024. 1, 2, 1, 3, 3.2, 3.2, 3, 4, 1, 4.3, 5.2, 5.2, 5.2, A, B, B, 1, **??**, **??**, 5, 4, B, C
48. Jung Yeon Hwang, Stanislaw Jarecki, Taekyoung Kwon, Joohee Lee, Ji Sun Shin, and Jiayu Xu. Round-reduced modular construction of asymmetric password-authenticated key exchange. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 485–504, September 2018. 1
49. Jung Yeon Hwang, Stanislaw Jarecki, Taekyoung Kwon, Joohee Lee, Ji Sun Shin, and Jiayu Xu. Round-reduced modular construction of asymmetric password-authenticated key exchange. In Dario Catalano and Roberto De Prisco, editors, *Security and Cryptography for Networks - 11th International Conference, SCN 2018, Amalfi, Italy, September 5-7, 2018, Proceedings*, volume 11035 of *Lecture Notes in Computer Science*, pages 485–504. Springer, 2018. 2, 2.3, 2.3
50. Ren Ishibashi and Kazuki Yoneyama. Compact password authenticated key exchange from group actions. In Leonie Simpson and Mir Ali Rezazadeh Baee, editors, *Information Security and Privacy - 28th Australasian Conference, ACISP 2023, Brisbane, QLD, Australia, July 5-7, 2023, Proceedings*, volume 13915 of *Lecture Notes in Computer Science*, pages 220–247. Springer, 2023. 1

51. Gembu Ito, Akinori Hosoyamada, Ryutaroh Matsumoto, Yu Sasaki, and Tetsu Iwata. Quantum chosen-ciphertext attacks against feistel ciphers. In Mitsuru Matsui, editor, *Topics in Cryptology - CT-RSA 2019 - The Cryptographers' Track at the RSA Conference 2019, San Francisco, CA, USA, March 4-8, 2019, Proceedings*, volume 11405 of *Lecture Notes in Computer Science*, pages 391–411. Springer, 2019. 3.1
52. Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? *IACR Cryptol. ePrint Arch.*, page 324, 2024. 1, 1, 2.2, 3, 3.1
53. Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486, April / May 2018. 3
54. Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 456–486. Springer, 2018. 6
55. Shaoquan Jiang and Guang Gong. Password based key exchange with mutual authentication. In Helena Handschuh and M. Anwar Hasan, editors, *Selected Areas in Cryptography, 11th International Workshop, SAC 2004, Waterloo, Canada, August 9-10, 2004, Revised Selected Papers*, volume 3357 of *Lecture Notes in Computer Science*, pages 267–279. Springer, 2004. 2.2
56. Jonathan Katz, Rafail Ostrovsky, and Moti Yung. Efficient password-authenticated key exchange using human-memorable passwords. In Birgit Pfitzmann, editor, *Advances in Cryptology - EUROCRYPT 2001, International Conference on the Theory and Application of Cryptographic Techniques, Innsbruck, Austria, May 6-10, 2001, Proceeding*, volume 2045 of *Lecture Notes in Computer Science*, pages 475–494. Springer, 2001. 2.2
57. Jonathan Katz and Vinod Vaikuntanathan. Smooth projective hashing and password-based authenticated key exchange from lattices. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 636–652. Springer, 2009. 2.1, 1, 2.2, 2.2
58. Jonathan Katz and Vinod Vaikuntanathan. Round-optimal password-based authenticated key exchange. *J. Cryptol.*, 26(4):714–743, 2013. 1, 2.2, 2.2
59. Michael Luby and Charles Rackoff. How to construct pseudorandom permutations from pseudorandom functions. *SIAM J. Comput.*, 17(2):373–386, 1988. 3.1
60. You Lyu and Shengli Liu. Hybrid password authentication key exchange in the UC framework. *IACR Cryptol. ePrint Arch.*, page 1630, 2024. 1, 1, 3, 3.2, 3.2, 2, 5.2
61. You Lyu, Shengli Liu, and Shuai Han. Efficient asymmetric PAKE compiler from KEM and AE. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part V*, volume 15488 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2024. 1, 3, 1, 2, 2.3, 3, 4, 3.3, 5.3, A, D

62. You Lyu, Shengli Liu, and Shuai Han. Universal composable password authenticated key exchange for the post-quantum world. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI*, volume 14656 of *Lecture Notes in Computer Science*, pages 120–150. Springer, 2024. 1, 2.1, 1, 2.2
63. Philip MacKenzie. The pak suite: Protocols for password-authenticated key exchange. *Contributions to IEEE P*, 1363(2), 2002. 2.3
64. Varun Maram and Keita Xagawa. Post-quantum anonymity of Kyber. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 3–35. Springer, Cham, May 2023. 1, 3.1
65. Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-n OT from programmable-once public functions. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 425–442. ACM, 2020. 1, 1, 2.2, 3.1
66. Ian McQuoid and Jiayu Xu. An efficient strong asymmetric PAKE compiler instantiable from group actions. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part VIII*, volume 14445 of *Lecture Notes in Computer Science*, pages 176–207. Springer, 2023. 3.3
67. National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report Federal Information Processing Standards Publications (FIPS PUBS) 203, U.S. Department of Commerce, Washington, D.C., 2024. 1, 1, 3.1
68. Jiaxin Pan and Runzhi Zeng. A generic construction of tightly secure password-based authenticated key exchange. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 143–175. Springer, 2023. 1
69. Jiaxin Pan and Runzhi Zeng. A generic construction of tightly secure password-based authenticated key exchange. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part VIII*, volume 14445 of *Lecture Notes in Computer Science*, pages 143–175. Springer, 2023. 1, 2.2
70. Sarvar Patel. Number theoretic attacks on secure password schemes. In *1997 IEEE Symposium on Security and Privacy, May 4-7, 1997, Oakland, CA, USA*, pages 236–247. IEEE Computer Society, 1997. 2.2
71. Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In David A. Wagner, editor, *Advances in Cryptology - CRYPTO 2008, 28th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2008. Proceedings*, volume 5157 of *Lecture Notes in Computer Science*, pages 554–571. Springer, 2008. 2.1
72. Peixin Ren, Xiaozhuo Gu, and Ziliang Wang. Efficient module learning with errors-based post-quantum password-authenticated key exchange. *IET Inf. Secur.*, 17(1):3–17, 2023. 2.2
73. Bruno Freitas Dos Santos, Yanqi Gu, and Stanislaw Jarecki. Randomized half-ideal cipher on groups with applications to UC (a)pake. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques,*

Lyon, France, April 23-27, 2023, Proceedings, Part V, volume 14008 of *Lecture Notes in Computer Science*, pages 128–156. Springer, 2023. 1, 2.1, 1, 2.2, 2.2, 2.2

74. Victor Shoup. Security analysis of spake2+. In Rafael Pass and Krzysztof Pietrzak, editors, *Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part III*, volume 12552 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2020. 1, 2.3, 2.3, 2.3, 4, 4.1, 4.2, 4.3, 4.4, 5.3, A
75. Dominique Unruh. Towards compressed permutation oracles. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part IV*, volume 14441 of *Lecture Notes in Computer Science*, pages 369–400. Springer, 2023. 3.1
76. Keita Xagawa. Anonymity of NIST PQC round 3 KEMs. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 551–581. Springer, Cham, May / June 2022. 1, 3.1
77. Yingshan Yang, Xiaozhuo Gu, Bin Wang, and Taizhong Xu. Efficient password-authenticated key exchange from RLWE based on asymmetric key consensus. In Zhe Liu and Moti Yung, editors, *Information Security and Cryptology - 15th International Conference, Inscrypt 2019, Nanjing, China, December 6-8, 2019, Revised Selected Papers*, volume 12020 of *Lecture Notes in Computer Science*, pages 31–49. Springer, 2019. 2.2
78. Mark Zhandry. How to record quantum queries, and applications to quantum indifferentiability. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268. Springer, 2019. 3.1
79. Jiang Zhang and Yu Yu. Two-round PAKE from approximate SPH and instantiations from lattices. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part III*, volume 10626 of *Lecture Notes in Computer Science*, pages 37–67. Springer, 2017. 2.1
80. Jiang Zhang, Zhenfeng Zhang, Jintai Ding, Michael Snook, and Özgür Dagdelen. Authenticated key exchange from ideal lattices. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 719–751. Springer, 2015. 2.2

A Detailed descriptions of the ideal functionalities

In this section we formally specify different variants of a secure PAKE notion in the universal composability framework. We refer to Section 4 for an overview of our definitional approach, and in particular to Figure 1 for the relations between some the variants we propose to the standard UC PAKE model $\mathcal{F}_{\mathsf{pwKE}}$ of Canetti et al. [26] and the *lazy extraction* UC PAKE model $\mathcal{F}_{\mathsf{lePAKE}}$ of Abdalla et al. [1]. We note that in Appendix B we formally show that our notions, resp. $\mathcal{F}_{\mathsf{PAKE}}$ and $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ defined below, are implied by these standard notions.

We use figures A1-A5 below to define the following PAKE-related functionalities:

- $\mathcal{F}_{\mathsf{PAKE}}$ consists of interface A1 and A4, where the latter *includes* text in blue, but *excludes* text in pink box and in teal.
- $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ is as $\mathcal{F}_{\mathsf{PAKE}}$ but adding interface A5, where the latter *includes* text in blue and in yellow box but *excludes* text in gray box and in teal.
- $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$ is as $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ but interface A5 is changed to *include* text in gray box and *exclude* text in yellow box.
- PAKEs which reveal password equality on passively connected sessions (RPE-PCS), denoted $\mathcal{F}^*_{\mathsf{PAKE}}$, $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{LE}}}$, and $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$, are defined as $\mathcal{F}_{\mathsf{PAKE}}$, $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$, and $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$, except interface A4 is modified to *include* text in pink box.
- $\mathcal{F}_{\mathsf{aPAKE}}$ and $\mathcal{F}^*_{\mathsf{aPAKE}}$ are defined as $\mathcal{F}_{\mathsf{PAKE}}$ and $\mathcal{F}^*_{\mathsf{PAKE}}$ except (1) interface A1 is replaced by interface A2, (2) the functionality is expanded by adding interface A3, (3) interfaces A4 and A5 are modified to *include* text in teal.
- $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA}}$ is defined as $\mathcal{F}^*_{\mathsf{aPAKE}}$ except interfaces A4 and A5 are modified to *include* text in red and *exclude* text in blue.[7]
- $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA}^{\mathsf{R}}}$ is defined as $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA}}$ except we add interface A5 s.t. it *includes* text in teal but *excludes* text in blue.

Definitional Choices: Discussion. As overviewed in Section 4, our starting point are the UC PAKE definition given by Shoup [74]. We chose them because for their expressivity, i.e. because they explicitly require the ideal-world adversary to know whether it utilizes a successful online password attack, via the `corrupt-key` query, or it interrupts a session, via the `random-key` query, or it lets two sessions passively connect, via the `fresh-key` and `copy-key` queries.

However, we modified Shoup's definition in several aspects. First, the original definition supported a more relaxed notion of matching sessions, somewhat akin to the recent UC *Bare PAKE* notion of Barbosa et al. [12]. We opted for the option used in the 'standard' UC PAKE model $\mathcal{F}_{\mathsf{pwKE}}$ of Canetti et al. [26], where sessions can match only if they run on the same session identifier ssid

[7] We do not formally define $\mathcal{F}_{\mathsf{PAKE\text{-}EA}}$ or $\mathcal{F}_{\mathsf{aPAKE\text{-}EA}}$, i.e. supporting EA without the RPE-PCS relaxation in either symmetric or asymmetric PAKE, but $\mathcal{F}_{\mathsf{PAKE\text{-}EA}}$ and $\mathcal{F}_{\mathsf{aPAKE\text{-}EA}}$ can be defined exactly as resp. $\mathcal{F}^*_{\mathsf{PAKE\text{-}EA}}$ and $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA}}$, except bit b is not sent to the adversary in response to the `passwords-matching` query.

and matching party and counterparty identifiers U, S, mostly to enable translation between protocol results in the standard setting and our notions. Secondly, whereas the original Shoup's model [74] leaked by default whether or not two matching sessions run on same passwords, we made this leakage optional, as captured by the `passwords-matching` query. Moreover, the `passwords-matching` query is not free: Once the adversary makes it the sessions change state from *original* to *eq-tested*(b) (where $b = 1$ if passwords match and otherwise $b = 0$), at which point the adversary can no longer stage an active attack on such sessions. This query therefore allows only to reveal password equality on *passively connected sessions*, i.e. RPE-PCS. In particular, the functionality preserves the original contract of Canetti et al. [26] that each session allows testing a single password, whether via an active attack or by an attempt to passively match two sessions.

These two changes essentially made the resulting notion $\mathcal{F}_{\mathsf{PAKE}}$ align with the Canetti et al.'s notion $\mathcal{F}_{\mathsf{pwKE}}$, and in Section B we argue that these two notions are equivalent. This equivalency allows us to utilize the PAKE results which target the UC PAKE model of Canetti et al. and/or its derivatives. In particular, it lets us utilize the results of Abdalla et al. on CPace [4], Arriaga et al. on NoIC/OQUAKE [9], Hesse and Rosenberg on the sequential hybrid combiner [47], and Gu [40] and Lyu et al. [61] on a PAKE-to-aPAKE compiler.

The definitions we adopt also include *protocol transcripts* as each party's final output together with the session key. We add transcripts because transcripts are utilized in the modular compositions constructions we use, specifically the PAKE combiner of [47] and the PAKE-to-aPAKE compiler of [40,61]. Note that we impose a strong requirement on what the transcript is: The only way two sessions (ssid, X, Y) and (ssid', X', Y') can output the same transcript is via the `fresh-key` and `copy-key` sequence, which can only be applied if $\mathsf{ssid} = \mathsf{ssid}'$, $X = Y'$, and $Y = X'$. Since in practice a man-in-the-middle adversary who controls the network can make any two parties have the same communication transcript, the notion of transcript we support is a so-called *full transcript*, denoted ftr, which in practice can be implemented as a concatenation, or a collision-resistant hash, of the communication transcript and the session,party,counterparty identifiers (ssid, P, P').

Finally, in the model for the asymmetric/augmented PAKE, $\mathcal{F}_{\mathsf{aPAKE}}$, we support interactive initialization, so the server doesn't see the user's password even at initialization. Lastly our notion explicitely requires globally unique identifiers rid per each account, which in practice can be implemented using a long-enough random salt value, and it treats the user account identifier uid, which is associated with a given user's account by the server, as a secret which is not publicly broadcast by either party.

A.1 Interfaces of symmetric and asymmetric PAKE functionality

Figure A1*: Initialization interface for symmetric PAKE*

User U starts a session with identifier ssid using (possibly mistyped) password pw.
On input $(\texttt{init-user-inst}, \mathsf{ssid}, S, \mathsf{pw})$ from party U:
- *Precondition:* Identifier ssid is used by no other user and by at most one server.
- Set the state of instance $(\mathsf{ssid}, U, S, \mathsf{pw})$ to *original*.
- Send $(\texttt{init-user-inst}, \mathsf{ssid}, U, S)$ to adversary $\mathcal{A}$.

Server S starts a session with identifier ssid and (possibly mistyped) password pw.
On input $(\texttt{init-server-inst}, \mathsf{ssid}, U, \mathsf{pw})$ from party S:
- *Precondition:* Identifier ssid is used by no other server and by at most one user.
- Set the state of instance $(\mathsf{ssid}, S, U, \mathsf{pw})$ to *original*.
- Send $(\texttt{init-server-inst}, \mathsf{ssid}, S, U)$ to adversary $\mathcal{A}$.

Figure A2*: Initialization interface for asymmetric PAKE (aPAKE)*

Changes in commands `init-user-inst` and `init-server-inst` compared to the symmetric PAKE interface are marked so .

User U initializes account $(\mathsf{rid}, \mathsf{uid})$ with server S.
On input $(\texttt{init-user}, \mathsf{rid}, S, \mathsf{uid}, \mathsf{pw})$ from user U:
- *Precondition:* No other user has been initialized with the same rid.
- Save record $(U, S, \mathsf{rid}, \mathsf{uid}, \mathsf{pw})$ and send $(\texttt{init-user}, U, S, \mathsf{rid})$ to adversary $\mathcal{A}$.

Server S initializes account $(\mathsf{rid}, \mathsf{uid})$ with user U.
On input $(\texttt{init-server}, \mathsf{rid}, U, \mathsf{uid})$ from server S:
- *Precondition:* No other server has been initialized with the same rid.
- Save record $(S, U, \mathsf{rid}, \mathsf{uid})$ and send $(\texttt{init-server}, S, U, \mathsf{rid})$ to adversary $\mathcal{A}$.

The adversary decides if the initialization succeeds.
On input $(\texttt{init-fin}, \mathsf{rid})$ from adversary $\mathcal{A}$:
- *Precondition:* $\exists$ U,S,uid,pw s.t. $\exists$ records $(U, S, \mathsf{rid}, \mathsf{uid}, \mathsf{pw})$ and $(S, U, \mathsf{rid}, \mathsf{uid})$.
- Save $(\texttt{pwdfile}, S, \mathsf{rid}, \mathsf{uid}, \mathsf{pw})$ and send output $(\texttt{init-fin}, \mathsf{rid})$ to S.

User U starts a session with identifiers rid, ssid.
On input $(\texttt{init-user-inst}, \mathsf{ssid}, S, \mathsf{rid}, \mathsf{uid}, \mathsf{pw})$ from user U:
- *Precondition:* Identifier ssid is used by no other user and by at most one server.
- Set the state of instance $(\mathsf{ssid}, U, S, \mathsf{rid}, \mathsf{uid}, \mathsf{pw})$ to *original*.
- Send $(\texttt{init-user-inst}, \mathsf{ssid}, U, S, \mathsf{rid})$ to adversary $\mathcal{A}$.

Server S starts a session with identifiers rid, ssid.
On input $(\texttt{init-server-inst}, \mathsf{ssid}, U, \mathsf{rid}, \mathsf{uid})$ from server S:
- *Precondition:* $\exists$ record $(\texttt{pwdfile}, S, \mathsf{rid}, \mathsf{uid}, [\mathsf{pw}])$.
- *Precondition:* Identifier ssid is used by no other server and by at most one user.
- Set the state of instance $(\mathsf{ssid}, S, U, \mathsf{rid}, \mathsf{uid}, \mathsf{pw})$ to *original*.
- Send $(\texttt{init-server-inst}, \mathsf{ssid}, S, U, \mathsf{rid})$ to adversary $\mathcal{A}$.

Figure A3: *Server compromise interface for asymmetric PAKE (aPAKE)*

Password file is compromised, allowing for offline password tests.
On input $(\texttt{compromise-pwdfile}, S, \mathsf{rid})$ from the environment $\mathcal{Z}$:
- *Precondition:* $\exists$ record $(\texttt{pwdfile}, S, \mathsf{rid}, \cdot, \cdot)$.
- Mark this record *compromised*, send $(\texttt{compromise-pwdfile}, S, \mathsf{rid})$ to adversary $\mathcal{A}$.

The environment allows the adversary to query the random oracle.
Input $(\texttt{oracle-query}, S, \mathsf{rid}, \mathsf{uid}', \mathsf{pw}')$ from the environment $\mathcal{Z}$:
- Send $(\texttt{oracle-query}, S, \mathsf{rid}, \mathsf{uid}', \mathsf{pw}')$ to adversary $\mathcal{A}$.

The adversary performs an offline password test.
On input $(\texttt{offline-test-pwd}, S, \mathsf{rid}, \mathsf{uid}', \mathsf{pw}')$ from adversary $\mathcal{A}$:
- *Precondition:* $\mathcal{Z}$ has previously called $(\texttt{oracle-query}, S, \mathsf{rid}, \mathsf{uid}', \mathsf{pw}')$.
- *Precondition:* $\exists$ record $(\texttt{pwdfile}, S, \mathsf{rid}, [\mathsf{uid}, \mathsf{pw}])$ marked *compromised*.
- If $(\mathsf{uid}', \mathsf{pw}') = (\mathsf{uid}, \mathsf{pw})$ send `correct` to adversary $\mathcal{A}$, else send `incorrect`.

Figure A4: *Adversary's interface*

Text in teal pertains only to *asymmetric PAKE (aPAKE)*.
Text in red pertains only to PAKE with entity authentication (PAKE-EA).
Including text in pink box defines $\mathcal{F}^*$ while excluding it defines $\mathcal{F}$.
Including text in pink box and text in red while excluding text in blue defines $\mathcal{F}^*$-type PAKE-EA.

The adversary learns whether passwords match on two passively-connected sessions.
On input $(\texttt{passwords-matching}, \mathsf{ssid}, U, S)$ from adversary $\mathcal{A}$:
- *Precondition:* Instance $U_{\mathsf{ssid}} = (\mathsf{ssid}, U, S, [\mathsf{rid}_U, \mathsf{uid}_U, \mathsf{pw}_U])$ has state *original*.
- *Precondition:* Instance $S_{\mathsf{ssid}} = (\mathsf{ssid}, S, U, [\mathsf{rid}_S, \mathsf{uid}_S, \mathsf{pw}_S])$ has state *original*.
- Set bit $b \leftarrow ((\mathsf{rid}_U, \mathsf{uid}_U, \mathsf{pw}_U) \stackrel{?}{=} (\mathsf{rid}_S, \mathsf{uid}_S, \mathsf{pw}_S))$.
- Change U_{ssid}'s state and S_{ssid}'s state to *eq-tested*(b) and send bit b to $\mathcal{A}$.

The adversary allows party X to terminate passively.
On input $(\texttt{fresh-key}, \mathsf{ssid}, X, \mathsf{ftr})$ from adversary $\mathcal{A}$:
- *Precondition:* $\exists$ instances $X_{\mathsf{ssid}} = (\mathsf{ssid}, X, [Y], ...)$ and $Y_{\mathsf{ssid}} = (\mathsf{ssid}, Y, X, ...)$.
- *Precondition:* X_{ssid} and Y_{ssid} have states *original* or *eq-tested*(b) for some b.
- *Precondition:* Full transcript ftr is globally unique.
- If $b = 1$ set $\mathsf{K} \leftarrow_R \mathcal{K}$, set X_{ssid}'s state to `fresh-key`, send $(\texttt{key}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ to X.
- Else set X_{ssid}'s state to `abort`, send $(\texttt{abort}, \mathsf{ssid})$ to X.

The adversary allows party Y to receive the same key if the passwords match.
On input $(\texttt{copy-key}, \mathsf{ssid}, Y)$ from adversary $\mathcal{A}$:
- *Precondition:* Inst. $Y_{\mathsf{ssid}} = (\mathsf{ssid}, Y, [X], \ldots, [\mathsf{pw}])$ has state *original* or *eq-tested*$(\cdot)$.
- *Precondition:* Inst. $X_{\mathsf{ssid}} = (\mathsf{ssid}, X, Y, \ldots, [\mathsf{pw}'])$ has state `fresh-key`.
- Retrieve $\mathsf{K}', \mathsf{ftr}'$ s.t. $(\texttt{key}, \mathsf{ssid}, \mathsf{K}', \mathsf{ftr}')$ was output to X_{ssid} and set $\mathsf{ftr} \leftarrow \mathsf{ftr}'$.
- If $\mathsf{pw} = \mathsf{pw}'$ set $\mathsf{K} \leftarrow \mathsf{K}'$, set Y_{ssid}'s state to `complete`, send $(\texttt{key}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ to Y.
- Else pick $\mathsf{K} \leftarrow_R \mathcal{K}$, set Y_{ssid}'s state to `complete`, send $(\texttt{key}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ to Y.
- Else set Y_{ssid}'s state to `abort`, send $(\texttt{abort}, \mathsf{ssid})$ to Y.

The adversary stages an online attack on party P, allowing one password guess.
On input $(\texttt{test-pwd}, \mathsf{ssid}, P, \mathsf{pw}')$ from adversary $\mathcal{A}$:
- *Precondition:* Instance $P_{\mathsf{ssid}} = (\mathsf{ssid}, P, \cdot, [\mathsf{rid}, \mathsf{uid}, \mathsf{pw}])$ has state *original*.

– If $\mathsf{pw}' = \mathsf{pw}$ change P_{ssid}'s state to *correct-guess* and send `correct` to $\mathcal{A}$.
– If $\mathsf{pw}' \neq \mathsf{pw}$ change P_{ssid}'s state to *incorrect-guess* and send `incorrect` to $\mathcal{A}$.

The adversary attempts to authenticate using the state of a compromised server.
On input $(\texttt{impersonate}, \mathsf{ssid}, U, S, \mathsf{rid})$ from adversary $\mathcal{A}$:
– *Precondition:* Instance $U_{\mathsf{ssid}} = (\mathsf{ssid}, U, S, \mathsf{rid}, [\mathsf{uid}, \mathsf{pw}])$ has state *original*.
– *Precondition:* $\exists$ record $(\texttt{pwdfile}, S, \mathsf{rid}, [\mathsf{uid}', \mathsf{pw}'])$ marked *compromised*.

– If $(\mathsf{uid}', \mathsf{pw}') = (\mathsf{uid}, \mathsf{pw})$ change P_{ssid}'s state to *correct-guess*, send `correct` to $\mathcal{A}$.
– Else change P_{ssid}'s state to *incorrect-guess*, send `incorrect` to $\mathcal{A}$.

The adversary chooses a key for a compromised instance.
On input $(\texttt{corrupt-key}, \mathsf{ssid}, P, \mathsf{K}^*, \mathsf{ftr})$ from adversary $\mathcal{A}$:
– *Precondition:* Instance $P_{\mathsf{ssid}} = (\mathsf{ssid}, P, \cdot, ...)$ has state *correct-guess*.
– *Precondition:* Full transcript ftr is globally unique.

– Change P_{ssid}'s state to `complete` and send output $(\texttt{key}, \mathsf{ssid}, \mathsf{K}^*, \mathsf{ftr})$ to P.

The adversary aborts an instance that has not reached completion.
On input $(\texttt{abort}, \mathsf{ssid}, P)$ from adversary $\mathcal{A}$:
– *Precondition:* Instance $P_{\mathsf{ssid}} = (\mathsf{ssid}, P, \cdot, ...)$ has not yet terminated.

– Change P_{ssid}'s state to `abort` and send output $(\texttt{abort}, \mathsf{ssid})$ to P.

The adversary allows party P to receive a random key.
On input $(\texttt{random-key}, \mathsf{ssid}, P, \mathsf{ftr})$ from adversary $\mathcal{A}$:
– *Precondition:* Instance $P_{\mathsf{ssid}} = (\mathsf{ssid}, P, \cdot, ...)$ has not yet terminated.
– *Precondition:* Full transcript ftr is globally unique.

– Change P_{ssid}'s state to `complete`.
– Sample $\mathsf{K} \leftarrow_R \mathcal{K}$ and send output $(\texttt{key}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ to P.

Figure A5*: Lazy extraction interface*

Including text in yellow box defines *lazy extraction* (lePAKE).
Including text in gray box defines *relaxed lazy extraction* (rlePAKE).
Text in teal pertains only to *asymmetric PAKE (aPAKE)*.
Text in blue is **excluded** in PAKE with entity authentication (PAKE-EA), in which case we call the interface *relaxed (rPAKE-EA)*.

The adversary registers a late password test against party P.
On input $(\texttt{register-test}, \mathsf{ssid}, P)$ from adversary $\mathcal{A}$:
– *Precondition:* Instance $P_{\mathsf{ssid}} = (\mathsf{ssid}, P, \cdot, ...)$ has state *original*.

– Flag instance P_{ssid} as tested, and change P_{ssid}'s state to *incorrect-guess*.

The adversary performs a late password test against party P.
On input $(\texttt{late-test-pwd}, \mathsf{ssid}, P, \mathsf{pw}')$ from adversary $\mathcal{A}$:
– *Precondition:* Instance $P_{\mathsf{ssid}} = (\mathsf{ssid}, P, \cdot, [\mathsf{rid}, \mathsf{uid}, \mathsf{pw}])$ has state `complete` or `abort`.
– *Precondition:* Instance P_{ssid} is flagged tested.

– Remove flag tested from P_{ssid}.
– (case i) if P_{ssid}'s state is `complete`, retrieve K output to P_{ssid} and:
  • if $\mathsf{pw}' = \mathsf{pw}$ set $\mathsf{K}' \leftarrow \mathsf{K}$ else sample $\mathsf{K}' \leftarrow_R \mathcal{K}$ set $\mathsf{K}' \leftarrow \perp$
  • send K' to $\mathcal{A}$
– (case ii) if P_{ssid}'s state is `abort` then:
  • if $\mathsf{pw}' = \mathsf{pw}$ then send `correct` to $\mathcal{A}$ else send `incorrect` to $\mathcal{A}$.

B Equivalences between PAKE functionalities

In this section we show that (1) any protocol that realizes the PAKE functionality $\mathcal{F}_{\mathsf{pwKE}}$ proposed by Canetti et al. [26] must also realize the PAKE functionality $\mathcal{F}_{\mathsf{PAKE}}$ we define, that (2) any protocol that realizes the lazy extraction version of the PAKE functionality $\mathcal{F}_{\mathsf{lePAKE}}$ proposed by Abdalla et al. [1] must also realize our lazy extraction PAKE functionality $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$, and that (3) any protocol that realizes the relaxed lazy extraction version of the PAKE functionality $\mathcal{F}_{\mathsf{rlePAKE}}$ introduced by Hesse and Rosenberg [47] must also realize our version of it $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$. Moreover, we argue that the reverse implications also hold, and therefore that these functionalities are pairwise equivalent.

However, for these equivalences to hold we must *amend* functionalities $\mathcal{F}_{\mathsf{pwKE}}$, $\mathcal{F}_{\mathsf{lePAKE}}$, and $\mathcal{F}_{\mathsf{rlePAKE}}$ in several ways explained below. We argue that these amendments should not invalidate any security proof that was done for their original variants, and we explain the nature of these amendments below. Our functionalities $\mathcal{F}_{\mathsf{PAKE}}, \mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}, \mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$ are defined in Appendix A, and our amended versions of the standard PAKE functionality $\mathcal{F}_{\mathsf{pwKE}}$, the lazy extraction PAKE functionality $\mathcal{F}_{\mathsf{lePAKE}}$ of [1], and the relaxed lazy extraction PAKE functionality $\mathcal{F}_{\mathsf{rlePAKE}}$ of [47], are shown in Figure 5. In Figure 5 we amend functionalities $\mathcal{F}_{\mathsf{pwKE}}$ of [26], $\mathcal{F}_{\mathsf{lePAKE}}$ of [1], and $\mathcal{F}_{\mathsf{rlePAKE}}$ of [47] in the following ways:

1. We apply to $\mathcal{F}_{\mathsf{pwKE}}$ the fix discussed e.g. by Abdalla et al. [4], see footnote 6, and which was already used in $\mathcal{F}_{\mathsf{lePAKE}}$ [1] and $\mathcal{F}_{\mathsf{rlePAKE}}$ [47].
2. Our variants of $\mathcal{F}_{\mathsf{pwKE}}$, $\mathcal{F}_{\mathsf{lePAKE}}$, and $\mathcal{F}_{\mathsf{rlePAKE}}$ output not only keys but also *protocol transcripts*, which are necessary to support the combiner shown in Protocol 3 and the PAKE-to-aPAKE compiler shown in Protocol 4, see Section 3. (Note that both constructions rely on the transcripts of the underlying PAKE to bind the upper-layer protocol to a PAKE instance.)
3. However, our treatment of transcripts is slightly non-standard in the following aspect: We effectively consider what we call a *full transcript*, denoted ftr, which can be implemented by concatenating the communication transcript with the session/party identifiers (ssid, U, S). This value is called a transcript only in its name and could be implemented using only e.g. 32 Bytes as a collision-resistant hash of (the concatenation of) the transcript and the identifier tuple (ssid, U, S).
   The security model $\mathcal{F}_{\mathsf{pwKE}}$ in Fig. 5 effectively imposes this constraint: The `NewKey` processing imposes a contract that two sessions can have a matching transcript only if they also have matching (ssid, P, P') and (ssid, P', P) identifiers (see the third bullet, which conditionally copies a key from one session to another). If parties do not include identifiers (ssid, P, P') in a protocol message then two sessions can share the same transcript without having matching identifiers, so another way to think of our model is that it forces adding these identifiers to the output transcript.
   Recall that in the UC setting identifiers ssid, P, P' are used as handles on a unique pair of sessions who are supposed to be each other's counterparties. If, in addition, these two sessions have the same communication transcripts then

Including text in blue and in yellow box defines *lazy extraction* $\mathcal{F}_{\mathsf{lePAKE}}$.
Including text in blue and in gray box defines *relaxed lazy extraction* $\mathcal{F}_{\mathsf{rlePAKE}}$.
Excluding the text in blue defines the standard PAKE functionality $\mathcal{F}_{\mathsf{pwKE}}$.
Changes from $\mathcal{F}_{\mathsf{pwKE}}$ of [26], $\mathcal{F}_{\mathsf{lePAKE}}$ of [1], and $\mathcal{F}_{\mathsf{rlePAKE}}$ of [47] are highlighted in teal (except for line marked (*) which is already present in [47]).

Initialization: Set $\mathsf{trSet} \leftarrow \{\}$

On $(\texttt{NewSession}, \mathsf{ssid}, P', \mathsf{pw}, \mathit{role})$ from party P:
- If this is the first or second query with matching ssid then save record $(\mathsf{ssid}, P, P', \mathsf{fresh}, \mathsf{pw})$ and send $(\texttt{NewSession}, \mathsf{ssid}, P, P', \mathit{role})$ to $\mathcal{A}$.

On $(\texttt{TestPwd}, \mathsf{ssid}, P, \mathsf{pw}')$ from adversary $\mathcal{A}$:
- Retrieve $\mathsf{rec} = (\mathsf{ssid}, P, \cdot, [\mathsf{flag}, \mathsf{pw}])$, s.t. $\mathsf{flag} = \mathsf{fresh}$, abort if no such record.
- If $\mathsf{pw}' = \mathsf{pw}$ then set $\mathsf{rec.flag} \leftarrow \mathsf{compromised}$ and send $\texttt{correct}$ to $\mathcal{A}$.
- If $\mathsf{pw}' \neq \mathsf{pw}$ then set $\mathsf{rec.flag} \leftarrow \mathsf{interrupted}$ and send $\texttt{incorrect}$ to $\mathcal{A}$.

On $(\texttt{NewKey}, \mathsf{ssid}, P, \mathsf{K}^*, \mathsf{ftr})$ from adversary $\mathcal{A}$:
- Retrieve $\mathsf{rec} = (\mathsf{ssid}, P, [P'], [\mathsf{flag}, \mathsf{pw}])$ s.t. $\mathsf{flag} \neq \mathsf{completed}$, abort if no such record
- (*) Abort if $\mathsf{flag} = \mathsf{fresh}$ but there is no record $(\mathsf{ssid}, P', P, ...)$.
- If $\mathsf{flag} = \mathsf{fresh}$ and $\exists$ record $\mathsf{rec}' = (\mathsf{ssid}, P', P, \mathsf{completed}, [\mathsf{pw}'])$ s.t. $\mathsf{rec}'.\mathsf{flag}$ was fresh when P' output $(\texttt{NewKey}, \mathsf{ssid}, \mathsf{K}', \mathsf{ftr}')$ s.t. $\mathsf{ftr}' = \mathsf{ftr}$ then set K as follows:
  - if $\mathsf{pw} = \mathsf{pw}'$ then set $\mathsf{K} \leftarrow \mathsf{K}'$,
  - if $\mathsf{pw} \neq \mathsf{pw}'$ then sample $\mathsf{K} \leftarrow_R \mathcal{K}$.
- Else if $\mathsf{ftr} \notin \mathsf{trSet}$ and $\mathsf{flag} = \mathsf{compromised}$ then set $\mathsf{K} \leftarrow \mathsf{K}^*$.
- Else if $\mathsf{ftr} \notin \mathsf{trSet}$ then sample $\mathsf{K} \leftarrow_R \mathcal{K}$.
- Else (i.e. if $\mathsf{ftr} \in \mathsf{trSet}$) abort processing this query.
- Output $(\texttt{NewKey}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ to P, add ftr to trSet, and set $\mathsf{rec.flag} \leftarrow \mathsf{completed}$.

On $(\texttt{Abort}, \mathsf{ssid}, P)$ from $\mathcal{A}$:
- If $\mathsf{rec} = (\mathsf{ssid}, P, \cdot, [\mathsf{flag}], \cdot)$ exists s.t. $\mathsf{flag} \neq \mathsf{completed}$ then set $\mathsf{rec.flag} \leftarrow \texttt{abort}$ and output $(\texttt{Abort}, \mathsf{ssid})$ to P.

On $(\mathsf{RegisterTest}, \mathsf{ssid}, P)$ from adversary $\mathcal{A}$:
- Retrieve $\mathsf{rec} = (\mathsf{ssid}, P, \cdot, [\mathsf{flag}], \cdot)$ s.t. $\mathsf{flag} = \mathsf{fresh}$, abort if no such record.
- Flag rec as tested, set $\mathsf{rec.flag} \leftarrow \mathsf{interrupted}$.

On input $(\mathsf{LateTestPwd}, \mathsf{ssid}, P, \mathsf{pw}')$ from adversary $\mathcal{A}$:
- Retrieve $\mathsf{rec} = (\mathsf{ssid}, P, \cdot, \mathsf{completed}, [\mathsf{pw}])$ marked tested, abort if no such record.
- Remove flag tested from rec and let K be the key output by P on session ssid.
- If $\mathsf{pw}' = \mathsf{pw}$ then set $\mathsf{K}' \leftarrow \mathsf{K}$ else sample $\mathsf{K}' \leftarrow_R \mathcal{K}$ set $\mathsf{K}' \leftarrow \perp$.
- Send K' to $\mathcal{A}$.

Fig. 5: The PAKE functionality $\mathcal{F}_{\mathsf{pwKE}}$ [26], the *lazy-extraction* PAKE functionality $\mathcal{F}_{\mathsf{lePAKE}}$ [1], and the *relaxed lazy-extraction* PAKE functionality $\mathcal{F}_{\mathsf{rlePAKE}}$ [47], amended by (1) exporting protocol transcripts, (2) preventing isolated parties from terminating in a fresh state [47], and (3) allowing aborts.

the adversary didn't interfere in their communication and the two sessions who intended to communicate proceeded to do so. Our terminology defines the two sessions as having the same *full transcript* if both conditions hold, i.e. (1) the sessions intended to communicate, i.e. their ssid, P, P' identifiers are matching, and (2) they did communicate without interference, i.e. their communication transcripts are also matching.

4. We use the fix recently proposed by Hesse and Rosenberg in $\mathcal{F}_{\mathsf{rlePAKE}}$ [47], and we adopt it to $\mathcal{F}_{\mathsf{pwKE}}$ and $\mathcal{F}_{\mathsf{lePAKE}}$. This fix prevents the functionality from terminating a session in a fresh state, i.e. a session which was not interfered with by the adversary, unless there is a *matching* session concurrently executed by some party. We call two sessions matching if they execute on the same ssid identifier and they have matching party/counterparty identifiers, e.g. if party X's intended counterparty is party Y and party Y's intended counterparty is party X. As argued by [47] this is not restrictive, i.e. a PAKE protocol cannot realize the original functionality with a simulator that terminates fresh sessions without a matching counterparty present. (Briefly speaking, a secure PAKE protocol cannot consist of a single X-to-Y protocol flow because it would be insecure against an offline dictionary attack which performs Y's code on any password candidate and compares the result to the session key which was output by Y.)
5. Since in the classical PAKE protocol CPace [44], see Protocol 2 in Section 3, an honest party can terminate with an abort, the version of symmetric PAKE functionalities we adopt must support this option as well.[8]
6. Finally, since we only argue security in the static setting we omit adaptive corruption commands in any functionalities. (Adaptive corruption queries are also omitted in many prior presentations of PAKE functionalities.)

Realizing our PAKE functionalities with standard UC PAKEs. If protocol Π realizes standard PAKE functionalities $\mathcal{F}_{\mathsf{pwKE}}$, $\mathcal{F}_{\mathsf{lePAKE}}$, or $\mathcal{F}_{\mathsf{rlePAKE}}$, then it implements an interface where an honest party P starts the protocol on inputs $(\texttt{NewSession}, \mathsf{ssid}, P', \mathsf{pw}, role)$, where P' is the identity of its intended counterparty, and outputs a tuple $(\texttt{NewKey}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ or $(\texttt{abort}, \mathsf{ssid})$ when the protocol terminates, see Fig. 5. Protocol Π' which realizes resp. functionality $\mathcal{F}_{\mathsf{PAKE}}$, $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$, or $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$, needs to conform to only a cosmetically different interface: Parties U and S playing respectively a user and a server role react to environment commands resp. $(\texttt{init-user-inst}, \mathsf{ssid}, S, \mathsf{pw})$ and $(\texttt{init-server-inst}, \mathsf{ssid}, U, \mathsf{pw})$ and their outputs are either $(\texttt{key}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ or $(\texttt{abort}, \mathsf{ssid})$, see $\mathcal{F}_{\mathsf{PAKE}}$ and $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ interfaces A1 and A4 in Appendix A. Consequently, if we denote the two values of field *role* are denoted as user and server then the translation between these protocols is only cosmetic, as shown in Figure 6.

[8] In CPace, see Protocol 2, party abort can be predicted from the transcript, which enables easy simulation of honest parties in the $\mathcal{F}_{\mathsf{lePAKE}}$ model extended by abort.

Party U: On input $(\texttt{init-user-inst}, \mathsf{ssid}, S, \mathsf{pw})$:
- Run protocol Π on inputs $(\texttt{NewSession}, \mathsf{ssid}, S, \mathsf{pw}, \mathsf{user})$.
- If Π outputs $(\texttt{NewKey}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ then output $(\texttt{key}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$.
- If Π outputs $(\texttt{Abort}, \mathsf{ssid})$ then output $(\texttt{abort}, \mathsf{ssid})$.

Party S: On input $(\texttt{init-server-inst}, \mathsf{ssid}, U, \mathsf{pw})$:
- Run protocol Π on inputs $(\texttt{NewSession}, \mathsf{ssid}, U, \mathsf{pw}, \mathsf{server})$.
- If Π outputs $(\texttt{NewKey}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$ then output $(\texttt{key}, \mathsf{ssid}, \mathsf{K}, \mathsf{ftr})$.
- If Π outputs $(\texttt{Abort}, \mathsf{ssid})$ then output $(\texttt{abort}, \mathsf{ssid})$.

Fig. 6: Protocol Π' which realizes functionality $\mathcal{F}_{\mathsf{PAKE}}$, resp. $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$, given a PAKE protocol Π that realizes functionality $\mathcal{F}_{\mathsf{pwKE}}$, resp. $\mathcal{F}_{\mathsf{lePAKE}}$, of Fig. 5.

Reduction arguments. We argue that the above simple protocol translation works, i.e. that if Π realizes $\mathcal{F}_{\mathsf{pwKE}}$ then Π' realizes $\mathcal{F}_{\mathsf{PAKE}}$, if Π realizes $\mathcal{F}_{\mathsf{lePAKE}}$ then Π' realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$, and if Π realizes $\mathcal{F}_{\mathsf{rlePAKE}}$ then Π' realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$.

Theorem 4. *Three statements hold about protocol Π' in Fig. 6: (1) If protocol Π realizes functionality $\mathcal{F}_{pwKE}$ then Π' realizes $\mathcal{F}_{PAKE}$; (2) If Π realizes $\mathcal{F}_{lePAKE}$ then Π' realizes $\mathcal{F}_{PAKE^{LE}}$; (3) If Π realizes $\mathcal{F}_{rlePAKE}$ then Π' realizes $\mathcal{F}_{PAKE^{RLE}}$. Functionalities $\mathcal{F}_{pwKE}, \mathcal{F}_{lePAKE}, \mathcal{F}_{rlePAKE}$ are defined in Figure 5 while $\mathcal{F}_{PAKE}, \mathcal{F}_{PAKE^{LE}}, \mathcal{F}_{PAKE^{RLE}}$ are defined in Appendix A.*

Proof. We provide a direct proof without game hopping. We formally argue part (2) of the theorem, and we note that parts (1) and (3) are argued in almost identical way.

We prove that for any environment $\mathcal{Z}$, there is a simulator $\mathcal{S}_{\Pi'}$ s.t. $\mathcal{Z}$'s interaction with $\mathcal{S}_{\Pi'}$ and functionality $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$, is identical to $\mathcal{Z}$'s interaction with protocol Π' in Fig. 6. Note that by the assumption that protocol Π UC-realizes $\mathcal{F}_{\mathsf{lePAKE}}$ there exists simulator $\mathcal{S}_{\Pi}$ s.t. the latter view is indistinguishable from $\mathcal{Z}$'s interaction with a system where subprotocol Π is replaced by simulator $\mathcal{S}_{\Pi}$ interacting with functionality $\mathcal{F}_{\mathsf{lePAKE}}$. Therefore, we construct simulator $\mathcal{S}_{\Pi'}$ as a composition of simulator $\mathcal{S}_{\Pi}$, which expects to interact with functionality $\mathcal{F}_{\mathsf{lePAKE}}$, and a "translator" simulator, which translates the $\mathcal{F}_{\mathsf{lePAKE}}$ queries of $\mathcal{S}_{\Pi}$ into $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ queries, and conversely translates $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ responses into messages that look like $\mathcal{F}_{\mathsf{lePAKE}}$ responses. We show the code of this translator simulator as Simulator 1, where the role of $\mathcal{S}_{\Pi}$ is played by ideal-world adversary $\mathcal{A}$.

Simulator 1 emulates $\mathcal{F}_{\mathsf{lePAKE}}$ perfectly given access to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$, i.e. the composition of $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ and Simulator 1 implements the same state machine that is defined by functionality $\mathcal{F}_{\mathsf{lePAKE}}$. This claim can be formally proven by in-lining the code of $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ into the code of Simulator 1 and then re-writing the resulting game until it matches $\mathcal{F}_{\mathsf{lePAKE}}$. In lieu of making such formal argument, we note the following points to justify this claim:

- When $\mathcal{Z}$ starts any party of protocol Π' then Simulator 1 sends the same `NewSession` message to $\mathcal{A}$ as $\mathcal{F}_{\mathsf{lePAKE}}$ would.

Simulator 1: A simulator that emulates $\mathcal{F}_{\mathsf{lePAKE}}$ to $\mathcal{A}$ on access to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$. (Text in blue is *excluded* if the simulator emulates $\mathcal{F}_{\mathsf{pwKE}}$ to $\mathcal{A}$ on access to $\mathcal{F}_{\mathsf{PAKE}}$.)

Initialization: Set trSet $\leftarrow \{\}$

On (`init-user-inst`,ssid,U,S) from $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
Store (ssid, U, S) marked fresh
Send (`NewSession`, ssid, U, S, user) to $\mathcal{A}$

On (`init-server-inst`,ssid,S,U) from $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
Store (ssid, S, U) marked fresh
Send (`NewSession`, ssid, U, S, user) to $\mathcal{A}$

On (`TestPwd`, ssid, P, pw′) from $\mathcal{A}$
Send (`test-pwd`, ssid, P, pw′) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
If $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ returns `correct` then:
- Mark (ssid, P, $\cdot$) as compromised
- Send `correct` to $\mathcal{A}$

If $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ returns `incorrect` then:
- Mark (ssid, P, $\cdot$) as interrupted
- Send `incorrect` to $\mathcal{A}$

On (RegisterTest, ssid, P) from $\mathcal{A}$
Send (`register-test`, ssid, P) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
If ∃ (ssid,P,$\cdot$) marked fresh mark it interrupted

On (LateTestPwd, ssid, P, pw′) from $\mathcal{A}$
Send (`late-test-pwd`, ssid, P, pw′) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
If $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ returns K′ then pass K′ to $\mathcal{A}$

On (`Abort`, ssid, P) from $\mathcal{A}$
Send (`abort`, ssid, P) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
If ∃ (ssid,P,$\cdot$) not marked completed mark it `abort`

On (`NewKey`, ssid, P, K*, ftr) from $\mathcal{A}$
Retrieve session (ssid, P, [P']) marked flag
Abort if flag ∈ {`abort`, completed}
Abort if flag = fresh but there is no ses. (ssid,P', P)
(i) If flag = compromised and ftr ∉ trSet:
- Send (`corrupt-key`, ssid, P, K*, ftr) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
- Mark (ssid, P, P') as completed

(ii) Else if flag = interrupted and ftr ∉ trSet:
- Send (`random-key`, ssid, P, ftr) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
- Mark (ssid, P, P') as completed

(iii) Else if (ssid,P',P) marked fresh and ftr ∉ trSet:
- Send (`fresh-key`, ssid, P, ftr) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
- Mark (ssid, P, P') as `fresh-key`(ftr)

(iv) Else if (ssid,P',P) marked `fresh-key`(ftr):
- Send (`copy-key`, ssid, P) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
- Mark (ssid, P, P') as completed

(v) Else if ftr ∉ trSet:
- Send (`random-key`, ssid, P, ftr) to $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$
- Mark (ssid, P, P') as completed

(vi) Else abort

– Note that session (sid, P, $\cdot$) transitions from state fresh to compromised or interrupted on $\mathcal{A}$'s query `TestPwd`, or to state interrupted on query RegisterTest, or to state `abort` on query `Abort`, exactly as it would in $\mathcal{F}_{\mathsf{lePAKE}}$. Further, these transitions happen if and only if $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ performs the corresponding transitions as well. Moreover, a session reacts to LateTestPwd query also as in $\mathcal{F}_{\mathsf{lePAKE}}$ because $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ responds to `late-test-pwd` with a key only if the session is in state completed and $\mathcal{A}$ queried RegisterTest before the session transitioned to completed.

– Simulator 1 together with $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ manages transcript inputs ftr in `NewKey` queries in the same way as $\mathcal{F}_{\mathsf{lePAKE}}$: Each ftr input $\mathcal{A}$ sends in `NewKey` queries must be unique except if the `NewKey` query pertains to session (ssid, P, P') which is fresh and for which there exists a counterparty session (ssid, P', P) which completed by going from state fresh to `fresh-key`(ftr), see case (iv) in `NewKey` processing in Simulator 1. Note that session (ssid, P', P) can get to state `fresh-key`(ftr) only if $\mathcal{A}$ calls (`NewKey`, ssid, P', P, $\cdot$, ftr) when this session was fresh, see case (iii). Note also that in this case $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ marked session (ssid, P', P) as `fresh-key`, in which case the simulator's (`copy-key`, ssid, P) query in case (iv) goes through, and $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ copies transcript ftr output by session (ssid, P', P) as the transcript which will also be output by session (ssid, P, P'). Functionality $\mathcal{F}_{\mathsf{lePAKE}}$ enforces the same contract on ftr uniqueness with the same exception as above.

- Note that if session $(\mathsf{ssid}, P, \cdot)$ is compromised or interrupted then subsequent $(\texttt{NewKey}, \mathsf{ssid}, P, \mathsf{K}^*, \cdot)$ query assigns output K of this session in the same way as $\mathcal{F}_{\mathsf{lePAKE}}$, i.e. $\mathsf{K} = \mathsf{K}^*$ if the session is compromised, see case (i), or K is random if the session is interrupted, see case (ii).
- If session (ssid, P, P') is fresh but there is no matching counterparty session (ssid, P', P) then the simulator ignores a $(\texttt{NewKey}, \mathsf{ssid}, P, ...)$ query, just like $\mathcal{F}_{\mathsf{lePAKE}}$ would. Finally, if this counterparty session exists but it is not marked fresh or $\texttt{fresh-key}(\mathsf{ftr})$, i.e. session (ssid, P', P) was actively attacked or aborted then $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ will assign this session a random key K, see case (v), just like $\mathcal{F}_{\mathsf{lePAKE}}$ would.

We note that part (3) of the theorem is argued identically, without changes to the code of Simulator 1, except in this reduction value K' passed by the simulator from functionality $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$ to adversary $\mathcal{A}$ in response to LateTestPwd query, could be a rejection sign $\mathsf{K}' = \perp$. Part (1) of the theorem is also argued in the same way, except that in this case the code of Simulator 1 can be simplified by removing the RegisterTest and LateTestPwd interfaces.

Realizing standard UC PAKEs with our PAKE functionalities. The implications also hold in the other direction, i.e. (1) any protocol that realizes our $\mathcal{F}_{\mathsf{PAKE}}$ also realizes the standard PAKE functionality $\mathcal{F}_{\mathsf{pwKE}}$ [26], (2) any protocol that realizes our $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ also realizes the standard lazy-extraction PAKE functionality $\mathcal{F}_{\mathsf{lePAKE}}$ [1], and (3) any protocol that realizes our $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$ also realizes the relaxed lazy extraction PAKE $\mathcal{F}_{\mathsf{rlePAKE}}$ introduced in [47]. (In all cases we assume that $\mathcal{F}_{\mathsf{pwKE}}$, $\mathcal{F}_{\mathsf{lePAKE}}$, and $\mathcal{F}_{\mathsf{rlePAKE}}$ refer to the *amended* versions of these functionalities defined in Fig. 5.) As in the other direction, the input/output interface needs to be cosmetically adjusted for this protocol translation, similarly as is done in Fig. 6, and the implications can be shown by a similar 'simulator translator' as the simulator shown above.

We do not formally show the simulator that emulates $\mathcal{F}_{\mathsf{PAKE}}$ given access to $\mathcal{F}_{\mathsf{pwKE}}$, but we argue that it has an easier task than the one above because it is given more expressive queries like `fresh-key`, `copy-key`, `corrupt-key`, `random-key`, and it needs to translate them to the simplified `NewKey`-only interface of $\mathcal{F}_{\mathsf{pwKE}}$. (The argument for translating $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ into $\mathcal{F}_{\mathsf{lePAKE}}$ and $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$ into $\mathcal{F}_{\mathsf{rlePAKE}}$ is identical.) Technically, the simulator will keep a state variable for each session, matching the state which functionality $\mathcal{F}_{\mathsf{PAKE}}$ would assign to this session. If the adversary makes an 'inadmissible' $\mathcal{F}_{\mathsf{PAKE}}$ query, e.g. `fresh-key` followed by `test-pwd`, the simulator uses this state to screen such queries out. If queries `fresh-key` and `copy-key` are 'admissible', the simulator queries `NewKey` to $\mathcal{F}_{\mathsf{PAKE}}$, otherwise it screens them out. Adversarial queries `test-pwd` and `abort` are passed to $\mathcal{F}_{\mathsf{pwKE}}$, query `corrupt-key` following `test-pwd` is translated into `NewKey`, while query `random-key` which hasn't been preceded by `test-pwd` is translated into two queries to $\mathcal{F}_{\mathsf{pwKE}}$: First `test-pwd` with a $\perp$ for a password guess, which guarantees that $\mathcal{F}_{\mathsf{pwKE}}$ marks its state for that session as interrupted, followed by `NewKey`.

C Security proofs for CPaceOQUAKE

Below we include the proofs of Theorem 1 and 2 of Section 5. These proof are either corollaries of the claims on the sequential PAKE combiner shown by Hesse and Rosenberg [47] or they follow by a simple modification of the proofs in [47].

Proof of Theorem 1.

Proof. The proof of this theorem follows by a simple modification of the proof of Lemma 1 in Hesse and Rosenberg [47].

Lemma 1 in [47] argues that the sequential combiner realizes functionality $\mathcal{F}_{\mathsf{rlePAKE}}$ if PAKE #1 realizes $\mathcal{F}_{\mathsf{lePAKE}}$ and PAKE #2 satisfies the (statistical) *pre-shared key equality-hiding* (PSK-EH) property. First, note that protocol 3 is a case of the sequential PAKE combiner of [47] applied to a CPace variant where the CPace output key K_1 is 'post-processed' using a KDF to derive keys $\mathsf{K}_1^L, \mathsf{K}_1^R$ from K_1. Clearly if KDF is a secure PRG and CPace realizes $\mathcal{F}_{\mathsf{lePAKE}}$ then the 'KDF-postprocessed CPace' used in protocol 3 also realizes $\mathcal{F}_{\mathsf{lePAKE}}$, therefore Lemma 1 of [47] applies to protocol 3 as well.

We first note that by Theorem 4 in Appendix B, any protocol that realizes $\mathcal{F}_{\mathsf{rlePAKE}}$ also realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$, and by inspection of the proof of Lemma 1 in [47], the proof can be translated into our definitional framework, to argue that if PAKE #1 realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ and PAKE #2 is PSK-EH than protocol 3 realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{RLE}}}$.

Since in our case PAKE #2 is OQUAKE, and does not satisfy PSK-EH (see Section 1), the above claim does not apply, and we need to argue a modified claim, namely that if PAKE #1 realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ then for *arbitrary* PAKE #2 it holds that protocol 3 realizes functionality $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$.

However, by inspection of the proof in Lemma 3 in [47] (Lemma 3 in is used within the proof of Lemma 1 in [47]), essentially the same proof can be used to argue the modified claim above. Specifically, the only changes in the proof of Lemma 3 in [47] needed to argue the modified claim are as follows. First, the simulator in Figure 14 of [47] needs the following change in the last line of `NewKey` processing, which dictates how the simulator emulates honest party executing PAKE #2. In [47] this last line specifies that the simulator should run PAKE #2 on input Z (the same value is denoted $\overline{\mathsf{pw}}$ in protocol 3) s.t. (case 1) if the PAKE #1 session was successfully attacked via `TestPwd` on $\mathsf{pw}' = \mathsf{pw}$ then Z is computed as in the protocol, via an RO hash on inputs known to the attacker, or (case 2) in any other case, which corresponds to PAKE #1 outputting a secure key to the honest party, the simulator picks Z as a random κ-bit string. The change we must make is to modify the choice of Z in case 2 as follows. If (ssid, P, P') is the session, party, and counterparty identifiers on the simulated session then:

- (Case 2a) If session (ssid, P, P') terminates PAKE #1 in a fresh state and there exists a matching counterparty session (ssid, P', P) which is still in fresh state, the simulator picks random Z as before but it also queries $(\texttt{passwords-matching}, \mathsf{ssid}, P, P')$ to $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$ and if $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$ returns bit

$b = 1$, which indicates that the two sessions run on matching passwords, the simulator saves this Z value.
- (Case 2b) If session (ssid, P, P') terminates PAKE #1 in a fresh state, and there exists a matching counterparty session (ssid, P', P) which has terminated in the fresh state, and, moreover $(\texttt{passwords-matching}, \mathsf{ssid}, P', P)$ query which the simulator sent to $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$ when session (ssid, P', P) terminated returned $b = 1$ then the simulator retrieves value Z it saved when session (ssid, P', P) and uses the same Z as input to PAKE #2 on session (ssid, P, P').
- (Case 2c) In any other case the simulator runs PAKE #2 on a random Z.

Note that this change in the simulation ensures that if an adversary passively terminates two matching sessions in PAKE #1, and the two sessions held the same password, in which case they would compute the same keys K_1, and hence also form the same PAKE #2 inputs Z (denoted $\overline{\mathsf{pw}}$ in protocol 3), then (by case 2a and 2b) the simulator runs PAKE #2 protocol for these two sessions on *the same* random Z value. However, in any other case, including sessions which were interrupted or they were passively terminated without a matching fresh session, or the matching session didn't run on the same password, the (by case 2c), the simulator runs PAKE #2 protocol for these two sessions on *independent* random Z values.

Indeed, by inspection of the proof of Lemma 3 in [47], the only change in the game transitions in that proof (and this is the second change to that proof we need to adjust it to the modified claim) is in game G9. The point of this game is to *randomize Z of honest parties with matching passwords* [47], and the modification this game change introduces in [47] is to make the Z value of session (ssid, P, P') in case 2b above independent from the Z value which was chosen for (ssid, P', P), and the distinguishing advantage this game change created is reducible to the distinguishing advantage against the PSK-EH property (denoted *PEH* in [47]). The modification needed for the proof of the modified claim is to eliminate this transition, and then in Game 11, which in [47] eliminates passwords (and password-equality testing) from the simulation, the modified Game 11 would syntactically replace the internal password-equality check with an external one, via a call to functionality $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$. The proof of Lemma 3 in [47] modified in this way is a proof of the modified claim we need, i.e. that if PAKE #1 realizes $\mathcal{F}_{\mathsf{PAKE}^{\mathsf{LE}}}$ then for any PAKE #2 protocol 3 realizes $\mathcal{F}^*_{\mathsf{PAKE}^{\mathsf{RLE}}}$.

Finally, we note that term $\frac{3}{2^\kappa}$ in the exact bound in Theorem 1 is the upper-bound on the real vs. ideal distinguishing advantage implied by the proof of Lemma 3 in [47] (and this bound is unaffected by the above modifications), while term $\mathrm{Adv}^{\mathrm{CPace}}_{\mathsf{lePAKE}}(\mathcal{A}_1) + \mathrm{Adv}^{\mathrm{KDF}}_{\mathsf{PRG}}(\mathcal{A}_2)$ is the distinguishing advantage against the $\mathcal{F}_{\mathsf{lePAKE}}$ notion of the 'KDF-postprocessed CPace', implied by a hybrid that builds either a reduction $\mathcal{A}_1$ against the $\mathcal{F}_{\mathsf{lePAKE}}$ notion of CPace or the reduction $\mathcal{A}_2$ against the PRG security of KDF, and both reductions run in time required by the CPaceOQUAKE adversary $\mathcal{A}'$ plus an additive overhead required to emulate the rest of the protocol.

Proof of Theorem 2.

Proof. Theorem 2 of Section 5 is a corollary of Lemma 2 of [47], and of Theorem 4 in Appendix B.

Lemma 2 of [47] says that if PAKE #1 has the *one-round perfect password-hiding* property [47] and PAKE #2 realizes $\mathcal{F}_{\mathsf{pwKE}}$, then the sequential combiner of [47] realizes $\mathcal{F}_{\mathsf{pwKE}}$. The first thing to observe is that the CPace variant effectively used in protocol 3, which consists of CPace followed by generation of $\mathsf{K}_1^L, \mathsf{K}_1^R$ via KDF on CPace output K_1, realizes the same *one-round perfect password-hiding* property: The property depends on the existence of CPace simulator which can derive honest party session keys on any password input. (This simulator works by embedding trapdoored values $G = g^{\alpha_{\mathsf{pw}}}$ in hash outputs $\mathtt{H}_{\mathbb{G}}(\mathsf{fsid}, \mathsf{pw})$ and setting an honest party's CPace contribution as $A = g^a$ [assume that U is the honest party], which allows the simulator to compute the DH function $Z = \mathsf{DH}(G, A, B^*) = (B^*)^{a/\alpha_{\mathsf{pw}}}$ for any CPace contribution B^* sent by the adversary.) Clearly, if the simulator can compute CPace output K_1 corresponding to any password, then it can also compute the corresponding outputs $\mathsf{K}_1^L, \mathsf{K}_1^R$ of the 'KDF-postprocessed CPace' used in protocol 3. Consequently, Lemma 2 of [47] applies to our variant of the sequential combiner, hence under the above assumptions protocol 3 realizes functionality $\mathcal{F}_{\mathsf{pwKE}}$.

Since by Theorem 4 in Appendix B, any protocol that realizes functionality $\mathcal{F}_{\mathsf{pwKE}}$ also realizes functionality $\mathcal{F}_{\mathsf{PAKE}}$ (after syntactic changes to input/output notation shown in Figure 6 in Appendix B), consequently under the stated assumptions protocol 3 also realizes functionality $\mathcal{F}_{\mathsf{PAKE}}$. Finally, the term $\frac{2}{2^\kappa}$ in the exact bound in Theorem 2 is the upper-bound on the real vs. ideal distinguishing advantage which follows from the proof of Lemma 2 in [47].

D Security proof for the PAKE-to-aPAKE compiler

In this version of the report we only overview of how the proof for the security of protocol CPaceOQUAKE+, i.e. the proof of Theorem 3, differs from the proofs in [40,61]. The full proof will be included in the forthcoming revision of this report. Here we will use the proof in [40] as a point of comparison, and we will briefly sketch the modifications one must make in this proof to adjust it to the proof of Theorem 3.

The proof in [40] concerns a variant of the same PAKE(+KEM)-to-aPAKE compiler, with the following main differences (we refer to protocol 4 for the terminology): (1) the compiler of [40] does not explicitly consider a salt value rid assigned to each account, (2) in the compiler of [40] the S-to-U transmission is an authenticated encryption of KEM ciphertext c encrypted under key K, whereas in protocol 4 this authenticated encryption is specifically implemented as (ec, h_1) where ec is a OTP encryption of c under (a key derived from) K, and h_1 is an RO hash on ec and K (and also on the symmetric PAKE transcript), (3) the U-to-S transmission is a MAC on the PAKE transcript computed under (a key derived from) the same key K, whereas in protocol 4 this MAC is specifically implemented using another RO hash, and (4) the same hash computes the final output key.

The underlying PAKE model and the target functionality considered in [40] are also slightly different in that (5) the UC aPAKE model considered in [40] treats the user account identifier uid as a public value whereas in $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ it is considered private, (6) our target model $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ supports interactive initialization, and (7) the underlying symmetric PAKE is assumed to realize $\mathcal{F}_{\mathsf{PAKE}}$ in [40] whereas in our case we weaken it to $\mathcal{F}^*_{\mathsf{PAKE^{RLE}}}$, i.e. a (Revealing Password Equality on Passively Connected Sessions) relaxed lazy-encryption PAKE, and consequently our target asymmetric PAKE model $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ is likewise a (RPE-PCS) relaxed PAKE (a variant of the lazy extraction which pertains to PAKEs with entity authentication).

However, none of these changes affect the underlying logic of the proof in [40]. Changes (2), (3), and (4) slightly simplify the argument because in the random oracle model the security of the RO-derived encryption and authentication is statistical and requires no reduction. Modification (1) is straightforward: It adds a unique public value to the inputs of the password hash $\mathtt{H}_{\mathsf{pb}}$ input. Its effect is primarily to replace the user identifier uid as a public handle on the user account at the server. This modification in turn allows the user identifier uid to be private. Indeed, in some parts of the proof we treat pair $(\mathsf{uid}, \mathsf{pw})$ as an effective password which the adversary must guess in an online or offline attack, which allows us to achieve property (5). Part (6) is simple to handle assuming secure initialization, i.e. assuming U and S are not corrupted at initialization and that the U-to-S communication in the initialization phase (i.e. transmission of the password file) proceeds on a U-S secure channel.

Finally, modification (7) also does not create significant difficulties: The $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ simulator which emulates the underlying symmetric PAKE functionality $\mathcal{F}^*_{\mathsf{PAKE^{RLE}}}$ passes the lazy-extraction queries $\mathsf{RegisterTest}$ and $\mathsf{LateTestPwd}$ to $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$, and replies `correct` or `incorrect` depending on whether the revealed key of the underlying $\mathcal{F}^*_{\mathsf{PAKE^{RLE}}}$ was used by the server in the computation of hash tag h_1 (which the real-world adversary can verify). We note that the $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ simulator does not use bit b from the `passwords-matching` query because whether $b = 1$ or $b = 0$ does not affect the simulation of values ec, h_1, h'_2 transmitted by the passively-connected honest parties in the aPAKE protocol, because either way they can be simulated as random bitstrings of appropriate size. However, we still need our target functionality $\mathcal{F}^*_{\mathsf{aPAKE\text{-}EA^R}}$ to have this RPE-PCS relaxation, because bit b is potentially revealed to the adversary in the underlying symmetric PAKE. (Note that the underlying symmetric PAKE realizes security model $\mathcal{F}^*_{\mathsf{PAKE^{RLE}}}$, hence it potentially reveals this bit.)